



**UNIVERSITÀ
DI TORINO**

Tecniche e Architetture Avanzate per lo Sviluppo del Software

Appunti del corso

Alessandro Zappatore

Università degli Studi di Torino
Anno accademico 2026/2027, primo semestre

Indice

1	Introduzione	3
1.1	Applicazioni mission critical	3
1.2	Enterprise Application Integration (EAI)	3
1.2.1	L'importanza dell'infrastruttura	4
1.2.2	Integrazione	4
1.3	Requisiti non funzionali	5
1.4	Modello a cascata (waterfall)	6
1.5	Metodologia Lean	6
1.5.1	Organizzazione a silos	7
1.5.2	L'EAI in questa fase: lo <i>spaghetti nightmare</i>	7
1.5.3	Tecnologia di rete	8
1.6	Agile	9
1.6.1	Evoluzione dei silos	9
1.6.2	Service-oriented architectures	9
1.6.3	Da Agile a oggi	9
1.7	Cloud come piattaforma	10
1.7.1	Modelli di servizio	11
1.8	Architetture orientate ai microservizi	11
1.8.1	Deployment	11
1.8.2	Ciclo di sviluppo	12
1.8.3	Cultura DevOps	12
1.8.4	Requisiti non funzionali promossi	13
2	Requisiti per architetture a microservizi	14
2.1	Serve un'architettura a microservizi?	14
2.1.1	Il monolite	14
2.1.2	L'inferno monolitico	14
2.1.3	SOA e microservizi a confronto	15
2.1.4	Microservizi e DevOps	15
2.1.5	Legge di Conway	17
2.2	Le sfide dei microservizi	17
2.3	Domain-Driven Design	17
2.3.1	Ubiquitous language	18
2.3.2	Modello di dominio	19
2.3.3	Quando usare il DDD	19
2.3.4	Mappa dei concetti del DDD	20
2.4	Pratiche di modellazione	20
2.4.1	Event storming	20
2.4.2	Value stream	21
2.4.3	User stories	21
2.4.4	CRC cards	22
2.5	DDD: Strategic vs Tactical	23
2.6	Strategic DDD	23
2.6.1	Sottodomini	24
2.6.2	Bounded context	25
2.6.3	Dai sottodomini ai bounded context	25
2.6.4	Interazioni tra bounded context	26
2.6.5	Shared Kernel e Anti-Corruption Layer	27
2.6.6	Front-end	28

2.6.7	Mockup	28
2.7	Tactical DDD	29
2.7.1	Entity	29
2.7.2	Value Object	30
2.7.3	Aggregate	30
2.7.4	Factory	31
2.7.5	Repository	32
2.7.6	Service	32
3	REST	34
3.1	Operazioni remote	34
3.1.1	Chiamata locale e chiamata remota	34
3.2	Operazioni remote su HTTP	35
3.2.1	Esempio di API su HTTP in stile RPC	35
3.3	REpresentational State Transfer (REST)	36
3.3.1	Risorse REST	37
3.3.2	Verbi REST	37
3.3.3	Status code delle risposte	39
3.4	REST vs RPC	40
3.5	Dove si usa REST nelle MSOA	41
3.5.1	API esterne e API interne	41

1 Introduzione

1.1 Applicazioni mission critical

Applicazione mission critical

Un'applicazione è **mission critical** quando le operazioni che permette di svolgere sono essenziali per il business di un'azienda, di un'istituzione, di un ente, ecc., dal punto di vista dell'organizzazione stessa.

Quanto un'applicazione sia mission critical si può misurare nella **perdita di denaro o di efficacia** che l'organizzazione subirebbe se l'applicazione smettesse di funzionare.

Esempi

- **Netflix**: se il sistema di streaming smettesse di funzionare, l'azienda perderebbe molti soldi, perché nessuno utilizzerebbe più la piattaforma.
- **Ospedale**: se si guasta il sistema di prenotazione delle sale di chirurgia, qualcuno potrebbe morire. Qui si passa al livello successivo: l'applicazione è **life critical**.

Proprio per questo assumono molta importanza i **requisiti non funzionali** (Sezione 1.3): quelli funzionali si danno per scontati, perché sono la base di partenza (*baseline*).

«FRs get you in the game, NFRs keep you alive in it.»

Cioè: i requisiti funzionali (FR) ti fanno entrare in gioco, quelli non funzionali (NFR) ti ci tengono vivo.

1.2 Enterprise Application Integration (EAI)

Enterprise Application Integration

Per **EAI** si intende l'insieme di tutti gli strumenti, le metodologie, i linguaggi, i pattern, ecc. che permettono di **condividere dati e processi** e di **coordinare** applicativi diversi che contribuiscono alla missione di una stessa organizzazione.

Le organizzazioni di cui parliamo sono tendenzialmente di medie/grandi dimensioni e hanno tutta una serie di dipartimenti che si occupano di aspetti diversi: tutti funzionali alla missione, ma con compiti abbastanza diversificati.

Quando negli anni '90 la digitalizzazione è arrivata nelle aziende, i vari dipartimenti si sono costruiti ciascuno il proprio software; a un certo punto, però, è diventato necessario **far parlare tra loro** questi software.

La Figura 1 mostra un esempio delle applicazioni di un'organizzazione, disposte su tre livelli: in alto quelle di interazione con il cliente (CRM, catalogo, carrello, pagamenti e fatturazione, assistenza clienti); in basso quelle di back office (ERP, gestione del magazzino e dei trasporti, contabilità, analytics e business intelligence); in mezzo quelle che fanno da ponte (gestione degli ordini, motore di raccomandazione).

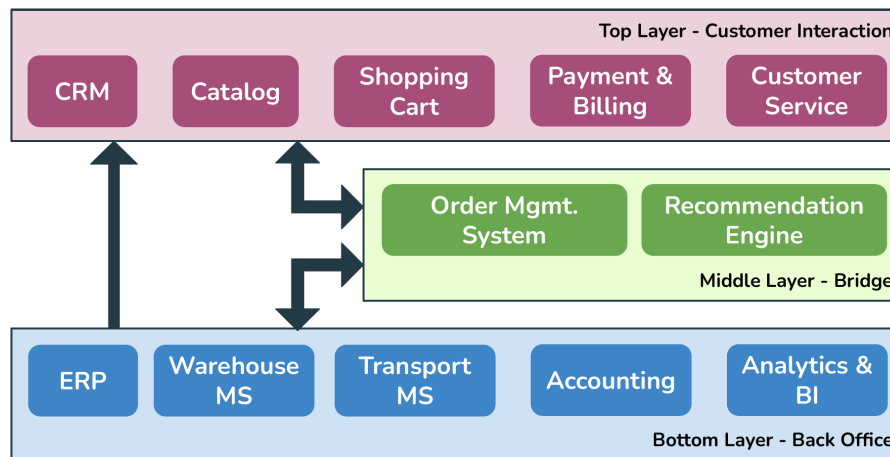


Figura 1: Esempio di applicazioni di un'organizzazione che devono comunicare tra loro

1.2.1 L'importanza dell'infrastruttura

Con il passare del tempo si è dovuto lavorare sulla comunicazione, e questo ha reso importante l'**infrastruttura**. Per questo motivo, nelle architetture a microservizi le applicazioni vengono costruite prevedendo già che avranno bisogno di comunicare.

È proprio l'importanza dell'infrastruttura ad aver fatto sì che venissero sviluppati, e resi quasi imprescindibili, strumenti come **Docker** e **Kubernetes**. Il loro compito è **modularizzare e virtualizzare l'infrastruttura**: rendendola flessibile e modulare, permettono allo sviluppatore di programmare a prescindere dall'infrastruttura.

Siamo abituati all'idea che ci sia il sistema operativo e, sopra, il software (o al limite il sistema operativo, la rete e, sopra, il software). Qui invece c'è uno **strato ulteriore**: qualcuno che si fa carico di gestire l'infrastruttura di comunicazione tra le varie parti dell'applicazione, e lo fa in maniera **laica** (Figura 2).

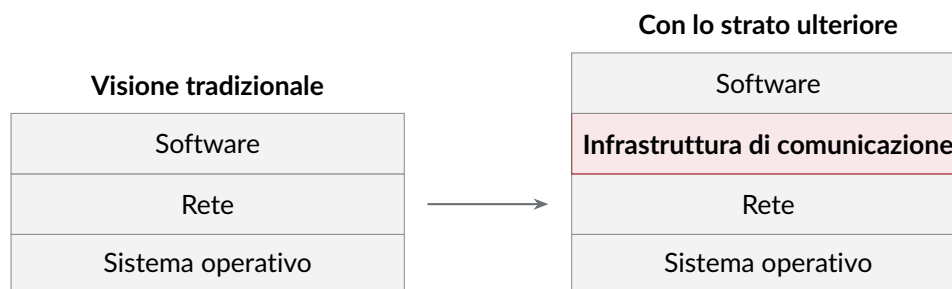


Figura 2: Lo strato ulteriore che si fa carico dell'infrastruttura di comunicazione (es. Docker e Kubernetes)

Infrastruttura «laica»

Nei sistemi antichi l'infrastruttura si faceva *ad hoc* per il sistema specifico.

L'idea nuova è quella di avere strumenti infrastrutturali che posso utilizzare su sistemi diversi, perché **generici e configurabili**: sono appunto *laici* rispetto al linguaggio di programmazione, al sistema operativo, alla rete, ecc.

1.2.2 Integrazione

Il tema dell'integrazione può essere scomposto in tre punti, che possono diventare più o meno un problema a seconda del tipo di applicazione che stiamo sviluppando.

Data integration Mi devo preoccupare della **consistenza dei dati** attraverso moduli diversi.

Esempio. Se ho un sistema di biblioteca e un sistema di prenotazione esami, devo garantire che i dati visti dai due sistemi siano gli stessi. Bisogna quindi stabilire chi possiede il database e chi comunica all'altro se c'è stata una variazione.

Process integration Riguarda come costruire un **processo operativo** (*workflow*) che richiede i servizi di elementi diversi.

Esempio. Uno studente si deve laureare. Un tempo il processo veniva svolto da un essere umano, che:

1. chiamava la segreteria studenti, che andava a vedere se lo studente aveva sostenuto tutti gli esami;
2. chiamava l'amministrazione e chiedeva se lo studente aveva pagato le tasse;
3. contattava il relatore di tesi per sapere se era d'accordo.

Per automatizzare tutto questo dobbiamo costruire qualcosa che sia in grado di interfacciarsi con tanti sistemi diversi, in un flusso logico e coerente, e che arrivi a emettere il suo verdetto.

Functional integration Ha più a che vedere con il **comporre i servizi** di moduli diversi, per poi «cucinarli» in qualche modo insieme e fornire all'utente un servizio nuovo.

Esempio. È ciò che avviene a livello di frontend: l'idea che tutti i servizi offerti da un'organizzazione possano avere un unico punto di raccolta, con accesso integrato per l'utente.

1.3 Requisiti non funzionali

I requisiti non funzionali più importanti sono i seguenti.

High availability / zero downtime Se l'applicazione è mission critical, deve essere sempre disponibile.

- **High availability:** dobbiamo fare in modo che un guasto in un pezzo del sistema impatti il meno possibile sul resto; dobbiamo quindi garantire una certa indipendenza tra i vari moduli.
- **Zero downtime:** se aggiorni l'applicativo, devo poterlo fare senza «tirarlo giù». Strumenti come Kubernetes permettono di sostituire una versione con un'altra senza interrompere il servizio.

Questo requisito è anche la prima motivazione per cui utilizzare Docker e Kubernetes.

Reliability L'applicazione deve funzionare correttamente.

Data consistency Se ci sono diversi moduli che usano gli stessi dati, devono essere informati se i dati cambiano.

Data integrity Una connessione di rete che cade non deve causare una corruzione dei dati.

Low latency Le risposte devono arrivare in tempi ragionevoli. Non riguarda solo l'utente, ma anche la comunicazione tra i moduli: quando i moduli comunicano tra loro, le latenze si moltiplicano.

Scalability L'applicazione deve essere in grado di gestire flussi che aumentano e diminuiscono nel tempo. Non voglio sovradimensionare il mio applicativo solo perché in un singolo giorno ho un picco di richieste (es. il Black Friday): voglio quindi che l'applicazione sia in grado di scalare in su e in giù a seconda delle necessità.

Security Autoesplicativo.

Resilience Capacità di non andare in crash alla prima cosa fuori programma. Per esempio, se c'è una momentanea assenza della rete, l'applicazione non deve «esplodere», e deve ritirarsi su quando la rete torna.

Maintainability Ha a che vedere con il lavoro degli sviluppatori: se dobbiamo aggiungere una feature o correggere un bug, non bisogna impazzire.

Observability Avere degli strumenti per osservare che cosa succede all'interno del sistema.

Auditability Certificazioni di qualità e strumenti di osservabilità che tracciano che cosa l'applicazione ha fatto e perché, i costi, ecc.

1.4 Modello a cascata (waterfall)

Inizialmente le applicazioni si sviluppavano con un approccio **a cascata**, cioè nello stesso modo in cui si sviluppavano le automobili: prima analizzo tutti i requisiti, poi progetto tutto il sistema, poi lo implemento, poi lo testo e infine lo do al cliente (Figura 3).

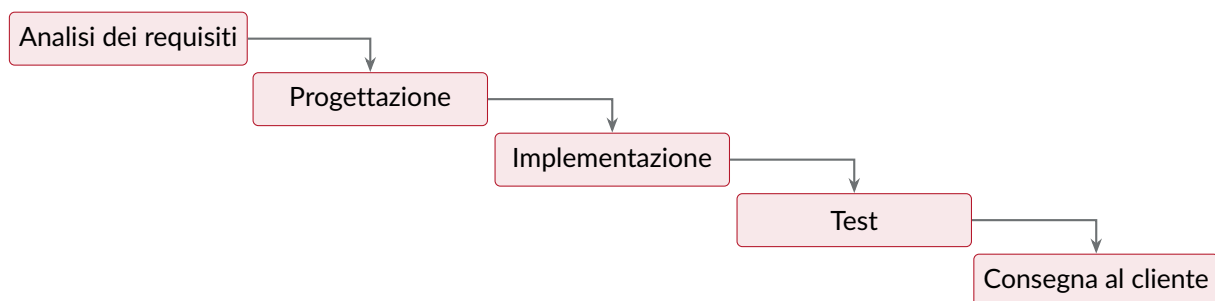


Figura 3: Le fasi del modello a cascata

Il problema è che solo alla fine posso scoprire che al cliente il risultato non va bene, o che ci sono degli errori. Queste cose le scopro tutte alla fine, quando il sistema è già tutto intero e complicato, per cui risolverle diventa un problema.

Questo problema della cascata non è stato osservato solo nell'industria del software, ma anche in altri tipi di industrie: tant'è che il primo tentativo di uscire dal modello a cascata, che va sotto il nome di metodologie **Lean** (snelle), nasce nell'industria dell'automobile.

1.5 Metodologia Lean

La metodologia Lean nasce in Giappone, alla **Toyota**, negli anni '50; negli anni '90 viene poi mutuata anche dall'industria dello sviluppo del software. Uno dei suoi strumenti è la **kanban** (in giapponese «insegna», «cartello»).

I principi della metodologia Lean consistono nell'eliminare **tre mali**:

Male	Nome	Che cosa significa
Spreco	<i>Muda</i>	Spreco di risorse e di tempo delle persone: non passare un sacco di tempo a fare cose che poi non servono.
Disequilibrio	<i>Mura</i>	Non avere picchi di produzione o di non produzione.
Sovraccarico	<i>Muri</i>	Si era capito che sfruttare e spremere troppo le persone, le macchine e le risorse non porta a una maggiore efficienza: si cerca quindi di dare un carico di lavoro adeguato.

1.5.1 Organizzazione a silos

Il lavoro era organizzato **a silos**: le varie attività (requisiti, sviluppo, testing e ops) erano tutte a compartimenti stagni. L'idea, proveniente dalla cascata, era la seguente:

1. prima lavorano quelli dei requisiti, che producono il loro materiale e lo mettono in mano agli sviluppatori;
2. gli sviluppatori sviluppano e danno il pacchetto a quelli del testing;
3. quelli del testing testano il tutto: se qualcosa non va lo ridanno agli sviluppatori, che correggono e lo ridanno al testing;
4. alla fine il tutto viene dato agli ops, che fanno il deploy.

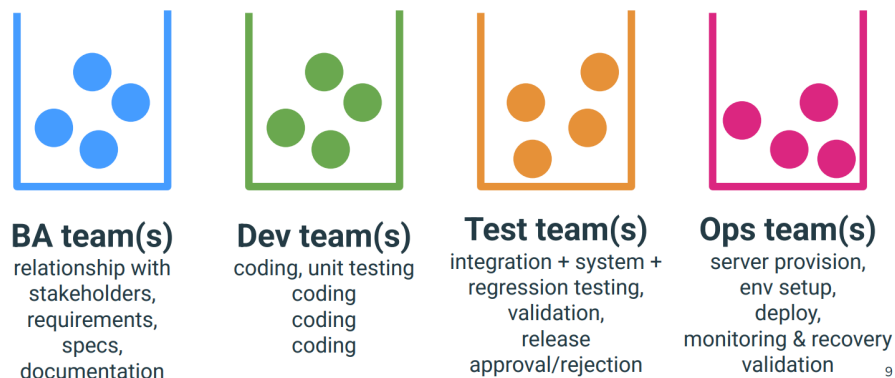


Figura 4: Organizzazione a silos

Il problema di questo sistema è che **gli ops non hanno la minima idea di quali fossero i requisiti**, perché non se ne sono occupati. Se alcuni dei requisiti impattano su questioni non funzionali che devono essere garantite proprio dagli ops, la faccenda si fa complicata.

1.5.2 L'EAI in questa fase: lo spaghetti nightmare

Dal punto di vista dell'EAI, in questa fase c'erano organizzazioni che accumulavano applicazioni che svolgevano i vari compiti, e si cercava di costruire tra di esse delle interazioni *ad hoc*; all'epoca «ad hoc» voleva dire che anche i protocolli erano differenti.

All'inizio degli anni '90 il problema comincia ad avere una dimensione tale da dover essere affrontato in maniera più organica: nasce lo **spaghetti nightmare** dell'EAI, dove ogni sistema ha un proprio database, una propria business logic e una propria UI, e tutti cercano di comunicare con tutti.

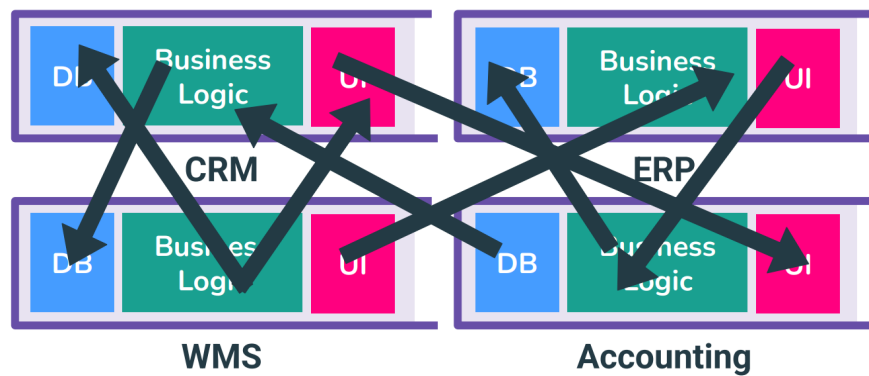


Figura 5: Esempio di *spaghetti nightmare*

In più, per ognuna di queste applicazioni le persone che ci lavorano sono divise in silos: il problema di comunicazione riguarda quindi anche gli esseri umani.

Qui nasce l'idea di costruire delle **piattaforme che facilitino l'infrastruttura (EAI platforms)**: applicativi che stanno in mezzo tra le applicazioni aziendali e che, attraverso lo sviluppo di funzioni logiche, fanno omogeneizzazione e comunicazione. Sono piattaforme che si situano a livello infrastrutturale ma (grossa differenza rispetto a quelle attuali) **conoscono il business** e i processi che devono avvenire all'interno dell'applicazione.

1.5.3 Tecnologia di rete

Dal punto di vista della tecnologia di rete, il percorso è stato il seguente:

1. **anni '70/'80:** networking fatto su misura;
2. **Ethernet e TCP/IP**, con protocolli *ad hoc* a livello applicativo: fino al livello di trasporto erano tutti d'accordo, sopra ognuno faceva quello che voleva;
3. **tra la fine degli anni '90 e i primi 2000:** HTTP si impone anche per le applicazioni.

La Figura 6 riassume su una linea del tempo tutto il periodo fino al 2000.

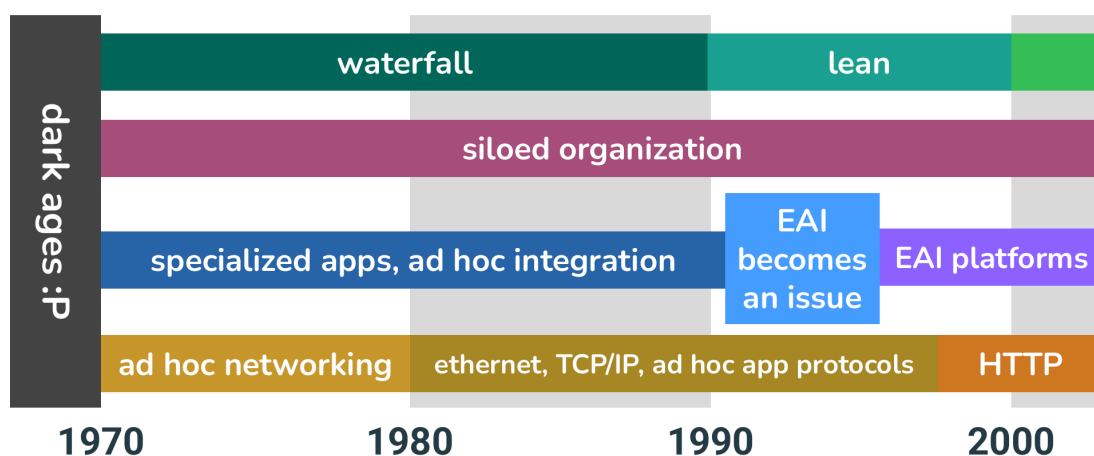


Figura 6: Dal 1970 al 2000: metodologie, organizzazione, integrazione e tecnologia di rete

1.6 Agile

A partire dagli anni 2000 nasce l'era **Agile**, con metodologie come XP, Scrum e Kanban, fino ad arrivare a Scrumban.

L'idea è quella di costruire le funzionalità dell'applicativo **a step**, compiendo ogni volta tutto il ciclo: in questo modo già dai primissimi cicli è possibile usare l'applicativo e vedere come funziona. Ho quindi la possibilità di avere un **feedback dai clienti** fin dalle primissime fasi, e non devo aspettare di avere tutto l'applicativo prima di ricevere un giudizio negativo che mi costringe a rifare tutto.

È quindi importante avere una **lista di priorità**, per sviluppare prima le funzioni critiche.

1.6.1 Evoluzione dei silos

L'organizzazione è ancora a silos, ma i silos si evolvono, perché i gruppi cominciano a unirsi un po': ad esempio, l'analisi dei requisiti comincia a entrare nel team di sviluppo.

In una struttura agile, nel team di sviluppo ci sono sia persone che seguono il lato requisiti, sia persone che fanno il testing, in modo da poter garantire **iteratività** e **incrementalità**. Restano ancora separati gli **ops**, incaricati del deployment e del monitoraggio dell'applicazione.



Figura 7: Evoluzione dell'organizzazione a silos

1.6.2 Service-oriented architectures

In questo periodo si impongono le **service-oriented architectures** e, con esse, l'idea di **servizio**: non penso più ad applicazioni separate che devo far parlare in qualche modo, ma ragiono già in principio in termini di servizi.

	Orientato a	Interfaccia
Servizio	business to business	<i>Application Programming Interface (API)</i>
Applicazione	utente	<i>User Interface (UI)</i>

In questa fase, per la comunicazione tra le applicazioni si tendeva a **costruire sopra il protocollo HTTP**, ad esempio con XML all'interno del corpo del messaggio HTTP: applicativi diversi potevano quindi utilizzare protocolli diversi.

1.6.3 Da Agile a oggi

Oggi tutto questo si evolve in tre elementi, a partire dal 2010 circa (Figura 8):

- il **modo di lavorare** si evolve nella cultura **DevOps**;

- le **architetture** si evolvono nelle **architetture a microservizi** (*Microservice Oriented Architectures*, MSOA);
- dal punto di vista della **tecnologia di rete** si va sulle **cloud-native applications**.

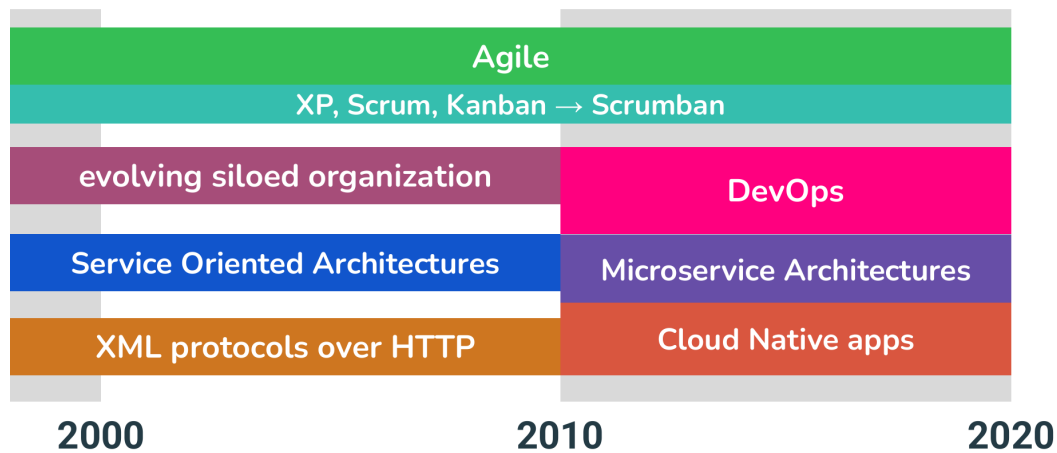


Figura 8: Dal 2000 al 2020: Agile, DevOps e cloud native

	Waterfall e Lean (fino al 2000)	Agile (dagli anni 2000)	Oggi (dal 2010 circa)
Modo di lavorare	Sviluppo a cascata, poi metodologie Lean. Silos a compartimenti stagni: requisiti, sviluppo, testing, ops.	Sviluppo a step, con feedback dei clienti fin dai primi cicli. Requisiti e testing entrano nel team di sviluppo; gli ops restano separati.	Cultura DevOps: sviluppatori e ops vengono fusi, i team sono orizzontali.
Architetture	Applicazioni separate con interazioni <i>ad hoc</i> (<i>spaghetti nightmare</i>), poi piattaforme che facilitano l'infrastruttura.	Service-oriented architectures.	Architetture a microservizi (MSOA).
Tecnologia di rete	Networking su misura, poi Ethernet e TCP/IP con protocolli applicativi <i>ad hoc</i> , infine HTTP.	Protocolli costruiti sopra HTTP (es. XML nel corpo del messaggio).	Cloud-native applications.

Tabella 1: Riepilogo dell'evoluzione nelle tre fasi

1.7 Cloud come piattaforma

L'idea è che il cloud diventi una **piattaforma**, tramite la quale avere una collezione (*pool*) di risorse **di rete, di memorizzazione e computazionali**, erogate come servizi e accessibili tramite API. Sopra questo pool c'è un'**astrazione**, uno strato simile a un sistema operativo, che mi permette di ignorare come esso sia costruito fisicamente: posso ignorare *dove* siano le risorse, *quali* siano, *come* siano collegate tra di loro e *quante* siano.

Questo strato che uniforma permette di installare l'applicazione nello stesso modo:

- su un singolo computer;
- su una rete locale fatta da qualche computer;

- su una rete distribuita di computer sparsi nel mondo.

Questo perché il cloud si pone come astrazione sopra le macchine fisiche e le reti.

Approccio dichiarativo

Una caratteristica del cloud è quella di garantire i servizi permettendo di esprimerli **dichiarativamente**: questo livello non viene programmato. Il sistemista non programma il cloud dicendogli come si deve comportare, ma **dichiara di quali risorse ha bisogno e a quale livello**, e quelle risorse vengono fornite.

1.7.1 Modelli di servizio

Il cloud si può configurare in modi diversi.

IaaS (Infrastructure as a Service) Viene venduto un insieme di risorse, tendenzialmente macchine virtuali, che posso usare come voglio: è come avere un computer in un posto remoto, che devo configurare come se fosse una macchina mia.

PaaS (Platform as a Service) Un provider può fornire un servizio singolo: ad esempio, si acquista uno spazio con Kubernetes già installato e lì si installa la propria applicazione. Usiamo cioè dei servizi nel cloud. Solitamente si paga in base alla quantità di utilizzo.

FaaS (Function as a Service) Posso installare non degli interi applicativi, ma delle singole funzioni, che vengono pagate per invocazione.

Tendenzialmente queste scelte sono:

- **on-demand**: permettono di pagare il servizio in base a quanto viene utilizzato;
- **ubiquitous**: posso spostare abbastanza facilmente la mia applicazione da un servizio all'altro.

A queste due caratteristiche si aggiungono l'**elasticity** (elasticità delle risorse) e il **metered use** (utilizzo misurato, a consumo).

1.8 Architetture orientate ai microservizi

Applicazioni cloud-native

Le applicazioni **cloud-native** sono applicazioni che hanno già nella loro progettazione originaria la predisposizione a questo tipo di ambiente per il futuro deployment.

Questo significa che sono strutturate in **piccoli moduli** che offrono servizi molto ben definiti e molto **incapsulati**, e che sono anche **poco accoppiati**, cioè non dipendono molto l'uno dall'altro.

1.8.1 Deployment

Dal punto di vista del deployment voglio che questi moduli possano girare in autonomia.

Containerizzati Ogni modulo gira in un ambiente di sua proprietà, quindi non si deve preoccupare di altri sistemi che interagiscono.

Orchestrati C'è una struttura che gestisce le relazioni e permette di comunicare.

Elastic scaling Proprio perché questi moduli sono indipendenti, se a un certo punto aumentano le richieste posso duplicarli.

Sono proprio le caratteristiche architetturali (servizi leggeri e poco accoppiati) e quelle di deployment, insieme, a definire le **architetture orientate ai microservizi**.

1.8.2 Ciclo di sviluppo

Le applicazioni cloud-native hanno delle caratteristiche anche rispetto al ciclo di sviluppo.

Continuous Integration Avendo tanti piccoli moduli, ciascuno con il proprio backend, bisogna poter testare ogni singolo modulo. Con i branch di Git posso sviluppare a parte e, una volta finito il modulo, portarlo alla composizione con il resto.

Continuous Delivery & Deploy Man mano che nuove feature vengono aggiunte al software, queste vengono rese attive per il pubblico.

Infrastructure as Code Permette alle applicazioni di girare su piattaforme come Docker e Kubernetes: all'interno del codice dichiaro che cosa mi serve dell'infrastruttura, così da far viaggiare il codice insieme ai suoi requisiti. È anch'essa dichiarativa: posso dire, ad esempio, «mi servono tra 1 e 5 copie che fanno auto-bilanciamento».

Dev/Test/Prod consistency Riguarda gli ambienti tipici dello sviluppo:

- **dev**: l'ambiente dove il software viene sviluppato (tipicamente la macchina dello sviluppatore);
- **test**: l'ambiente di testing, dove il software viene sottoposto a degli stress per vedere che cosa succede;
- **prod**: l'ambiente di produzione, dove il software viene rilasciato.

Shift-left tests/security Spostare i test e le analisi di sicurezza all'inizio del processo di sviluppo.

Baked-in observability Rendere osservabili gli eventi rilevanti di sistema, per riuscire a tracciare eventuali errori o malfunzionamenti. È *baked in* perché i meccanismi che rendono loggabili gli eventi sono già inseriti all'interno dell'applicazione, senza usare un servizio esterno.

1.8.3 Cultura DevOps

DevOps

Cultura, pratiche e strumenti per il ciclo di vita dello sviluppo software, basati sui principi Lean. L'idea centrale è che requisiti, sviluppo, test, sicurezza e QA lavorino insieme, con **un team per ogni servizio**.

Tutte queste caratteristiche del ciclo di sviluppo formano la cultura **DevOps**, in cui gli sviluppatori e gli ops vengono fusi. Gli ops fanno gli amministratori di sistemi Kubernetes, ma **non hanno bisogno di sapere che cosa fa il software che ci gira sopra**.

Tant'è che esistono tante piattaforme, come Microsoft Azure, Amazon AWS e Google Cloud, dove posso comprare uno spazio e configurare i container Kubernetes, con feature aggiunte dal proprietario della piattaforma, senza dover avere un system administrator.

I team non sono più verticali, in silos, ma **orizzontali**, e lavorano tutti insieme (Figura 9).

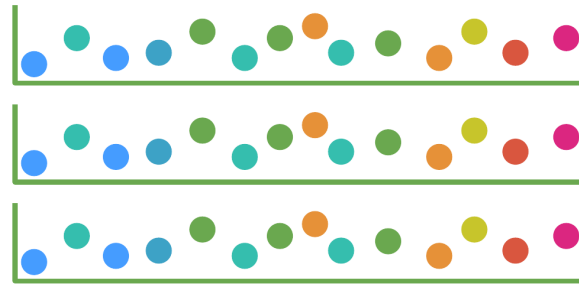


Figura 9: Team DevOps: competenze diverse nello stesso team, un team per ogni servizio

1.8.4 Requisiti non funzionali promossi

Tutto questo ha il fine di promuovere dei requisiti non funzionali, che sono una parte di quelli visti all'inizio (Sezione 1.3):

- **Portabilità:** possibilità di installare il mio software su una varietà di macchine.
- **Scalabilità:** elastic scaling.
- **Sicurezza.**
- **Mantenibilità.**
- **Evoluzione:** possibilità che l'applicativo si evolva nel tempo.
- **Alta disponibilità e affidabilità.**

In sintesi

- Un'applicazione è **mission critical** quando le sue operazioni sono essenziali al business dell'organizzazione; se è in gioco la vita delle persone è **life critical**. Per questo contano molto i **requisiti non funzionali**.
- L'**EAI** è l'insieme di strumenti, metodologie, linguaggi e pattern per condividere dati e processi e coordinare gli applicativi di una stessa organizzazione. L'integrazione ha tre aspetti: **data**, **process** e **functional integration**.
- L'**infrastruttura** è diventata centrale: Docker e Kubernetes la modularizzano e la virtualizzano in maniera *laica*, cioè generica e configurabile.
- Il modo di lavorare è passato da **waterfall** e **Lean** (con i silos) ad **Agile**, fino alla cultura **DevOps**; le architetture dallo *spaghetti nightmare* alle **SOA**, fino ai **microservizi** (Tabella 1).
- Il **cloud** è un pool di risorse dietro un'astrazione, configurato in modo **dichiarativo**; si offre come **IaaS**, **PaaS** o **FaaS**, in modo on-demand e ubiquitous.
- Le applicazioni **cloud-native** sono fatte di piccoli moduli incapsulati e poco accoppiati: containerizzati, orchestrati e con elastic scaling.

2 Requisiti per architetture a microservizi

L'analisi dei requisiti è molto importante nelle architetture a microservizi, perché non è detto che qualunque sistema software si possa, e si debba, realizzare con un'architettura a microservizi. Anzi: cercare di forzare un'architettura a microservizi su un tipo di applicativo che non ne beneficia in alcun modo è una perdita di tempo, una fatica inutile e uno spreco di risorse.

Perché sia necessario, utile e vantaggioso usare un'architettura a microservizi, il dominio applicativo deve prestarsi **per natura a essere diviso**.

2.1 Serve un'architettura a microservizi?

2.1.1 Il monolite

Monolite

Un'applicazione è un **monolite** quando tutte le sue funzionalità sono sviluppate e rilasciate come un'unica unità: un'unica base di codice, che viene compilata, installata e scalata tutta insieme.

L'architettura monolitica non è di per sé un male: il punto importante è la **dimensione** dell'applicazione. Tenzialmente la **soglia** è questa: l'applicazione deve poter essere gestita da un singolo team DevOps (tra le 5 e le 10 persone), perché in un monolite tutte le persone che ci lavorano devono avere una visione d'insieme.

Finché è piccolo, anzi, il monolite ha diversi vantaggi:

- è semplice da sviluppare;
- non è distribuito;
- non richiede test di integrazione complessi;
- il deployment è banale;
- non c'è accoppiamento, quindi i cambiamenti radicali sono facili;
- è facile da scalare.

2.1.2 L'inferno monolitico

Inferno monolitico

Si parla di **inferno monolitico** (*monolithic hell*) quando il monolite inizia a non andare più bene. È quello che succede quando un'applicazione nasce piccola, gestita da un unico team, ma poi il business cresce e diventa enorme: l'applicazione cresce, e le persone che ci lavorano diventano sempre di più e si dividono in più team.

A quel punto cominciano a vedersi le crepe:

- la **coordinazione** tra i vari team diventa difficile;
- nessuno ha più una **visione d'insieme** e riesce a capire l'intero sistema;
- ogni **cambiamento** apportato a un pezzo del sistema si ripercuote su altre parti, senza che qualcuno capisca che cosa sia veramente successo: i cambiamenti diventano rischiosi e imprevedibili;

- tutto deve stare sullo stesso **tech stack**, e quando questo diventa obsoleto l'aggiornamento è «tutto o niente»;
- la **scalabilità** non può riguardare il singolo componente: o ingrandisco tutto il sistema, o lo rimpicciolisco tutto;
- tutte le volte che devo fare il **deploy** devo fermare tutto e reinstallare l'applicazione: i rilasci sono lenti e causano downtime;
- se si pianta una funzione, **va giù tutto il sistema**.

2.1.3 SOA e microservizi a confronto

Rispetto alle service-oriented architectures viste nell'introduzione (Sezione 1.6.2), i microservizi si differenziano su tutti gli aspetti principali.

	SOA	Microservizi
Scope	Funzione di business di alto livello (grana grossa).	Sottodominio, singolo scopo (grana fine).
Comunicazione	<i>Smart pipes</i> (es. Enterprise Service Bus), protocolli pesanti.	<i>Dumb pipes</i> (es. message broker, oppure comunicazione diretta tra servizi con protocolli leggeri: REST, gRPC).
Storage	Database condivisi.	Un database per ogni servizio.
Deploy	Servizi strettamente accoppiati, che condividono lo stesso stack.	Servizi poco accoppiati e «poliglotti».
Scalabilità	Difficile, a causa delle dipendenze condivise.	Elevata: ogni servizio scala in modo indipendente.

Tabella 2: SOA e microservizi a confronto

2.1.4 Microservizi e DevOps

Il triangolo in Figura 10 mostra il legame tra tre elementi:

- in basso a destra, l'**architettura** dell'applicazione (a microservizi);
- in basso a sinistra, la struttura dell'**organizzazione** di cui la nostra applicazione deve supportare il business (team piccoli, autonomi e cross-funzionali);
- in alto, il **processo** di sviluppo (DevOps, continuous delivery e deployment).

Queste tre cose possono formare un **circolo virtuoso**, se in qualche modo si somigliano: l'architettura abilita l'organizzazione, ed entrambe abilitano il processo. Il risultato, al centro, è un rilascio del software rapido, frequente e affidabile.

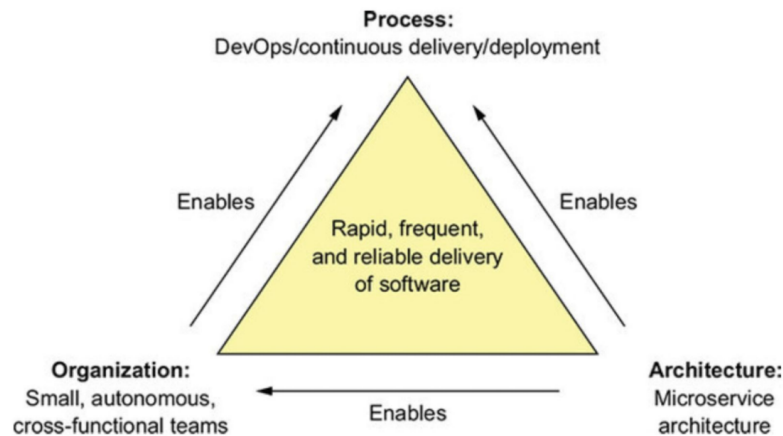


Figura 10: Microservizi e DevOps

La Figura 11 mostra un esempio di architettura a microservizi:

- l'**API Gateway** instrada verso i servizi le richieste che arrivano dalle applicazioni mobili;
- i servizi corrispondono a **capacità di business**, cioè ai sottodomini del Domain-Driven Design;
- ogni servizio espone delle **API**;
- i **dati** di un servizio sono privati.

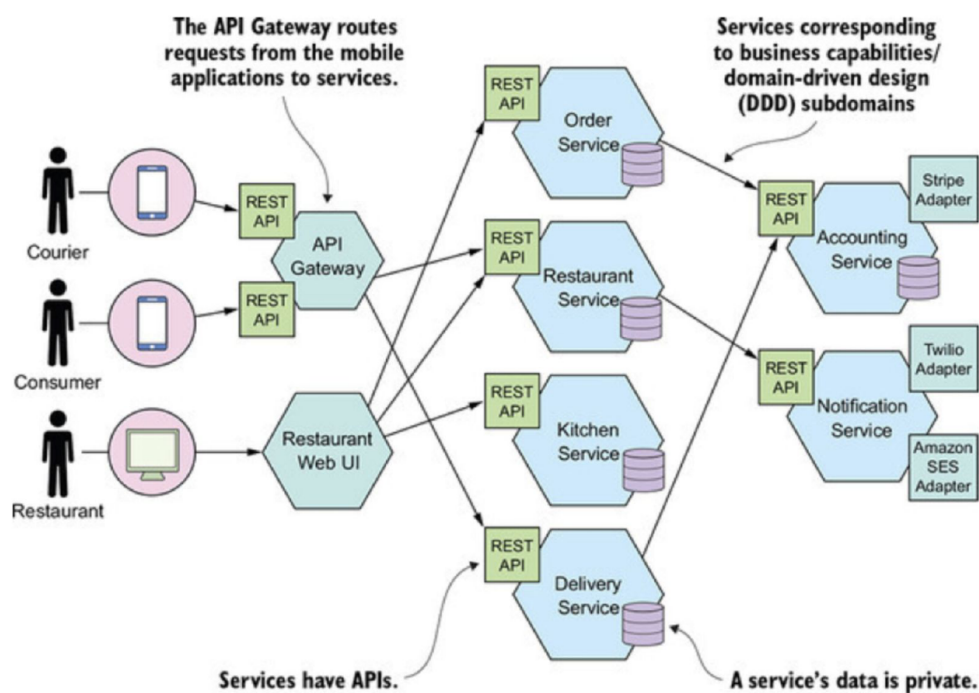


Figura 11: Esempio di architettura a microservizi

2.1.5 Legge di Conway

Legge di Conway

Le organizzazioni che progettano sistemi finiscono sempre con il produrre progetti che copiano le strutture comunicative dell'organizzazione stessa.

— Melvin Conway

Da questa legge è stata ricavata la **manovra inversa di Conway** (*Inverse Conway Maneuver*): se pensi di beneficiare di un'architettura a microservizi, ristruttura la tua organizzazione in modo che la sua struttura rispecchi quella dell'architettura. Così facendo, i team di sviluppo saranno poco accoppiati tanto quanto i servizi.

2.2 Le sfide dei microservizi

Mettere in piedi un'architettura a microservizi comporta diverse sfide.

Complessità È pur sempre un'architettura distribuita, con tutti i problemi che ne derivano.

Ristrutturazione organizzativa L'organizzazione che la mette in campo può doversi ristrutturare, e deve impegnarsi a farlo.

Prestazioni di rete Diventa rilevante il numero di interazioni necessarie per portare a termine un'operazione. Bisogna quindi:

- evitare troppe comunicazioni avanti e indietro (servizi «chiacchieroni», *chatty services*);
- usare operazioni «bulk», a grana grossa;
- minimizzare l'impatto della latenza.

Rete inaffidabile e servizi non disponibili L'architettura va studiata in modo che possa gestire la possibilità che un servizio vada giù, evitando che l'indisponibilità si propaghi **a cascata**.

Transazioni Se ogni microservizio ha un proprio database, quasi sempre dovremo rendere consistenti i dati tra database diversi. Nasce quindi il problema delle transazioni: garantire che una serie di modifiche venga fatta tutta oppure per niente, per mantenere la coerenza interna. Dovendolo fare su database diversi, però, non c'è un DBMS che garantisca la transazionalità. Il **CAP theorem** dimostra che in un'architettura distribuita non si possono avere tutte insieme consistenza (*Consistency*), disponibilità (*Availability*) e tolleranza al partizionamento della rete (*Partition tolerance*).

Sicurezza Va gestito servizio per servizio: ogni servizio deve verificare la sicurezza della richiesta che riceve (lo si fa tramite JWT). Affidarsi esclusivamente all'API gateway è ingenuo.

2.3 Domain-Driven Design

Un **modello di dominio** può essere visto come una mappa concettuale, orientata allo sviluppo del software, del dominio di lavoro dell'azienda che vogliamo supportare: per costruirlo devo studiare il dominio del business.

Domain-Driven Design (DDD)

Il **Domain-Driven Design** è un approccio alla progettazione del software che mette al centro il **modello di dominio**, il quale descrive in maniera formale ma concettuale i concetti, le entità, le relazioni e le regole che caratterizzano lo specifico dominio.

«Il Domain-Driven Design è un approccio allo sviluppo del software che incentra lo sviluppo sulla programmazione di un modello di dominio che abbia una ricca comprensione dei processi e delle regole di quel dominio.»

— Martin Fowler

Un modello di dominio ben costruito e progettato è l'**output** della fase di analisi dei requisiti e l'**input** della fase di progettazione, sia a livello architetturale sia a livello di codice. Sta in mezzo tra il business e il software, ed evolve insieme a entrambi.

Siamo sempre in un **contesto agile**, quindi iterativo e incrementale: se ho un business molto grande non faccio il modello di tutto il business, ma procedo per gradi. All'inizio faccio una sgrossatura di alto livello, per capire se il dominio si adatta a una suddivisione in microservizi e per operare questa suddivisione; poi ciascun team procede a suo modo, in maniera iterativa e incrementale, sul proprio microservizio.

Per questo il DDD è particolarmente adatto alle architetture a microservizi e alla cultura e alle pratiche DevOps.

2.3.1 Ubiquitous language

Per prima cosa il DDD definisce quello che si chiama *ubiquitous language*. Si parte dal **dominio del problema** (*problem domain*), dove ci sono le sfide del business, gli esperti di dominio e i requisiti, e ci si deve spostare verso il **dominio della soluzione** (*solution domain*), dove ci sono gli sviluppatori, che scrivono il codice di un sistema software che, sperabilmente, risponde ai requisiti e permette di affrontare le sfide di quel business.

L'idea è che ci sia un linguaggio *ubiquo*: i due domini devono usare la stessa lingua. Le cose si devono chiamare nello stesso modo in tutti e due i posti, così che non ci sia nessuna confusione sul significato dei termini.

Ubiquitous language

È un linguaggio comune, **costruito insieme e condiviso** dagli esperti di dominio e dal team di sviluppo. È costituito da:

- **entità di business**: gli oggetti rilevanti (es. nel dominio università: studenti, docenti, esami, corsi, lezioni, aule, ecc.);
- **regole di business** (es. uno studente si può laureare solo se ha sostenuto un certo numero di esami);
- **processi di business**: chi fa che cosa, e in che ordine;
- **eventi di business**: gli eventi rilevanti (es. lo studente che dà un esame, l'inizio dei corsi, le lauree).

Il linguaggio ubiquo deve essere utilizzato:

- nel glossario;
- in tutta la documentazione, nei diagrammi, ecc.;

- nel codice (nomi di classi, funzioni, moduli, ecc.).

Ubiquitous language e modello di dominio si costruiscono **allo stesso tempo**, partendo dal problem domain e usando alcuni strumenti: user stories, mockup e CRC cards. Siccome lo si fa in modo iterativo, ognuno di questi elementi può dare un feedback per migliorare gli altri (Figura 12).

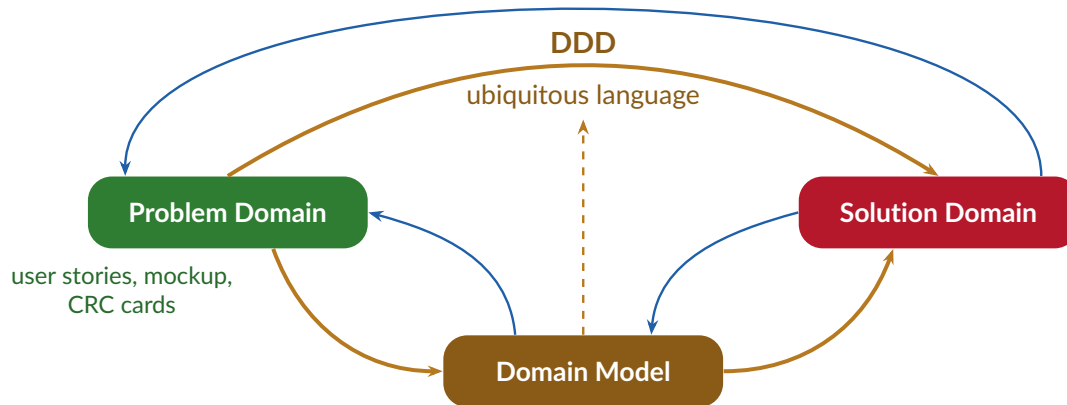


Figura 12: Dal problema alla soluzione: in arancione il percorso del DDD, in blu i feedback dell'approccio agile

2.3.2 Modello di dominio

Un modello di dominio cattura, in modo formale ma concettuale, i concetti, le entità, le relazioni e le regole rilevanti per uno specifico business.

Non possiamo aspettarci che un modello di dominio rifletta perfettamente il dominio reale di quel business: se ci provassimo, correremmo il rischio di non riuscire mai a terminarlo e di impelagarci in un tentativo perfezionistico, che va contro la metodologia agile. Il modello di dominio è una rappresentazione **selettiva**, non completa, del dominio del business: guardiamo il dominio attraverso la **prospettiva del problema** che stiamo cercando di risolvere.

«Tutti i modelli sono sbagliati, ma alcuni sono utili.»

Noi vogliamo cercare di farne uno utile. Gli aspetti da curare sono tre.

Astrazione Dobbiamo decidere a che livello di dettaglio e di granularità presentare le entità e le relazioni che ci sono all'interno del nostro dominio. Le entità rilevanti le faremo emergere dalle user stories.

Confini Teoricamente, da qualunque business io parta posso arrivare a qualsiasi altro business: devo quindi definire che cosa è pertinente per me e che cosa non lo è. Andremo anche a cercare dei confini *all'interno* del dominio: è questo che fa capire se il dominio è suddivisibile in microservizi.

Shared understanding Bisogna arrivare a una comprensione, sia del dominio sia del problema in oggetto, che sia condivisa tra esperti di dominio e sviluppatori.

2.3.3 Quando usare il DDD

Ha senso usare il DDD quando ci sono tre condizioni:

- un **dominio di business ben preciso** (*core domain*);
- una **complessità** elevata;
- una prospettiva di **evoluzione a lungo termine**.

2.3.4 Mappa dei concetti del DDD

La Figura 13 è la mappa di navigazione del DDD di Eric Evans: mette in relazione i concetti del Domain-Driven Design, molti dei quali vengono ripresi nel resto del capitolo.



Figura 13: Mappa di navigazione del DDD (Eric Evans, *Domain-Driven Design*)

2.4 Pratiche di modellazione

2.4.1 Event storming

Se ci dobbiamo confrontare con un dominio reale, una delle tecniche che si possono usare è l'**event storming**: una tecnica di modellazione collaborativa che serve a esplorare un dominio, a mettersi d'accordo su che cosa succede davvero nel business e a far emergere una prima bozza di ubiquitous language. È utile per allineare il linguaggio tra figure diverse (stakeholder del business, esperti di dominio, sviluppatori, designer, ops).

Si procede così:

1. le persone, tutte insieme (sviluppatori compresi), descrivono gli **eventi importanti** che possono succedere in quel dominio: ognuno li scrive su dei post-it e li attacca al muro in ordine sparso. Niente gergo tecnico, lettere grandi, verbi al passato, come se l'evento fosse già avvenuto (es. ordine effettuato, spedizione consegnata, esame superato);
2. i post-it vengono poi messi in **ordine cronologico** (che cosa deve succedere prima di che cosa?), per creare dei **flussi di eventi** (*stream*).

Questa specie di lavagna di post-it, con i suoi flussi di eventi, serve per confrontarsi e capire le entità del business, gli eventi rilevanti e come funzionano i processi all'interno del business. Si usa quindi per far comprendere i processi aziendali anche ai DevOps, e per iniziare a sviluppare l'ubiquitous language.

In questa fase si cominciano anche a notare sovrapposizioni o ambiguità nel linguaggio: alcune si possono chiarire, altre corrispondono a terminologie realmente diverse, usate da dipartimenti diversi del business.

2.4.2 Value stream

Value stream

Un **value stream** è la sequenza completa (*end-to-end*) delle attività tramite le quali l'organizzazione riesce a fornire valore al suo cliente, o a uno stakeholder.

Un value stream:

- cattura il **percorso completo**, dall'inizio alla fine;
- usa un **linguaggio di business**, comprensibile da tutti gli attori principali;
- mostra **quale valore** viene prodotto;
- è centrato sul punto di vista dei clienti del business (o degli stakeholder), ma **in terza persona**: è l'organizzazione che spiega quali sono i passaggi chiave, dal primo approccio fino al valore consegnato.

I value stream possono essere accompagnati dai **support stream**. In un'azienda complessa alcuni elementi di valore non sono rivolti al cliente, ma si scambiano tra i reparti dell'azienda (valore *interno*): sono cose che ho bisogno che succedano per far funzionare il mio business.

Ciò che differenzia i due è che in un value stream il valore emerge direttamente dallo stream; in un support stream non emerge direttamente, ma senza di esso non si riuscirebbe a ottenerlo (es. la gestione interna del magazzino).

2.4.3 User stories

User story

Una **user story** è un breve racconto che descrive un processo o un obiettivo dal punto di vista di un attore ben preciso.

L'attore diventa il personaggio di una storia di finzione: gli do un nome e un cognome. La storia deve essere calata su un utente particolare, quindi è individuale, ma allo stesso tempo deve essere rappresentativa; può (e dovrebbe) nascere da interviste reali. Una user story deve catturare:

- le **motivazioni**: il perché la persona fa quelle cose;
- le **azioni** che effettivamente compie;
- gli **obiettivi**, cioè i risultati che si aspetta di ottenere dalle sue azioni.

Per scriverla scelgo un value stream e racconto il viaggio di uno specifico utente all'interno di quello specifico stream. Il value stream su cui concentrarmi lo scelgo cercandone uno che metta in luce dove si concentra il valore del mio business.

Dettagli «di colore»

Una user story può (e dovrebbe) includere anche dettagli che fanno solo colore. Non sono necessariamente inutili:

- possono dare indicazioni sui compromessi tra requisiti non funzionali, sulla progettazione della UX, ecc.;
- aiutano a immaginare la storia in modo più vivido, e magari a individuare requisiti meno ovvi;
- non sempre sappiamo fin dall'inizio che cosa è «solo colore» e che cosa conta davvero.

Esiste anche un altro tipo di user story, le cosiddette **user stories agile**: storie molto piccole, di una sola frase, usate dai team agili per costruire la lista degli elementi del backlog. Seguono lo schema «Come [attore], voglio [obiettivo], così che [motivo]».

Esempio. «Come cliente voglio prenotare il cibo, così che quando arrivo sia già pronto.»

Dalla storia alle attività

La descrizione narrativa di una user story può poi essere scomposta in una lista di **attività** di una riga ciascuna (*breakdown* in attività), in cui si evidenziano:

- l'**attore** (chi): non il personaggio, ma il suo ruolo (es. «Cliente», «Cameriere»);
- l'**azione** (il verbo);
- il **risultato**, o *subgoal*, dell'azione (che cosa).

Non ci si concentra sulle azioni microscopiche: si descrivono quelle significative, che modificano lo stato di un'entità o di un processo nel dominio di business.

2.4.4 CRC cards

CRC è l'acronimo di *Class, Responsibility, Collaborator*. Le CRC cards sono un primo modo per descrivere le entità rilevanti all'interno del nostro sistema: sono delle schede che catturano quali sono le entità più rilevanti e le loro relazioni.

Possono essere introdotte in diverse fasi del processo: partono come bozze e si evolvono nel tempo, fino a diventare dei veri e propri diagrammi UML. In questa prima fase sono oggetti di **brainstorming**: l'obiettivo è far uscire i termini principali del nostro ubiquitous language, cioè le cose di cui si parla in questo dominio.

Tradizionalmente a ogni tipo di entità corrisponde una scheda, e questo ha senso per come le schede vengono usate dopo. Il sostituto digitale delle schede è un file condiviso, che tutto il gruppo può vedere.

Classe È il tipo di entità, cioè la classe *concettuale* (es. l'ordine del ristorante).

Responsabilità Che cosa le entità di questo tipo **conoscono** o **fanno** all'interno del sistema (es. l'ordine conosce l'elenco dei piatti e delle bevande scelti dai clienti, e tiene traccia del proprio stato e degli oggetti ordinati).

Collaboratori Le altre entità con cui l'entità **interagisce** per adempiere alle proprie responsabilità.

Il processo Le CRC cards si sviluppano in due passi.

1. **Identificare** le entità rilevanti di una user story, anche utilizzando il breakdown in attività. Il protagonista, cioè il soggetto che compie le azioni, non viene considerato un'entità rilevante. Se però compie delle azioni su altri personaggi, questi contano: a quel punto sono l'oggetto dell'azione, non il soggetto (sono gli attori di supporto, i «personaggi non giocanti» della storia). Per ciascuna entità si prepara velocemente una card, con il nome e le responsabilità che emergono dalla storia: che cosa conosce e che cosa fa.
2. Fare il **role-playing**: ciascun membro del team prende una o più entità e le interpreta, e insieme si attraversa la storia (se c'è il breakdown in attività, si può scorrere quella lista). Il role-playing fa emergere:
 - i **collaboratori**: si capisce quando due entità si devono parlare, cioè quando una deve chiedere qualcosa a un'altra per poter fare la sua parte;
 - le responsabilità mancanti o ambigue (chi fa, o chi sa, questa cosa?);
 - le entità mancanti.

Bisogna prestare attenzione alla possibilità che una CRC card diventi troppo «grassa», con troppe responsabilità e troppi collaboratori. Può voler dire due cose diverse:

- abbiamo aggregato troppo: in una sola entità ce ne sono due diverse, e la card si può dividere in due;
- siamo sul confine tra due sottodomini e stiamo mescolando i loro punti di vista: a volte una stessa entità viene vista in modi diversi in due dipartimenti diversi. In questo caso ce lo annotiamo, perché tornerà utile nel passo successivo.

2.5 DDD: Strategic vs Tactical

Nella sua filosofia, il Domain-Driven Design distingue due livelli: lo **Strategic DDD** e il **Tactical DDD**.

	Strategic DDD	Tactical DDD
Livello	Progettazione di alto livello.	Progettazione di basso livello.
Domanda	Come scomponiamo un dominio del problema articolato in sottoparti coerenti?	Come modelliamo l'interno di ciascuna sottoparte, in modo che rifletta fedelmente il dominio?
Parole chiave	Quadro d'insieme, architettura complessiva, modularizzazione, confini chiari.	Implementazione del modello di dominio, codice ben strutturato.

Strategic È di alto livello: guarda al problema nel suo complesso, cerca di capire il quadro d'insieme e come può essere modularizzato, cioè suddiviso in sottodomini con confini chiari.

Tactical È più spostato verso l'implementazione. Abbiamo già il dominio suddiviso, e possiamo passare a operazioni più raffinate: i contratti delle operazioni, una modellazione di dominio più raffinata, un modello di dominio più dettagliato. L'obiettivo è andare verso l'implementazione: devo tirare fuori *come* implementare il modello di dominio, quindi come sarà fatto il database, quali saranno le funzioni, gli oggetti e le classi (a seconda del linguaggio di programmazione scelto), e così via.

2.6 Strategic DDD

Nello Strategic DDD l'obiettivo è cercare di suddividere il dominio del sistema in sottodomini.

2.6.1 Sottodomini

Sottodominio

È un dipartimento o una capacità di business.

Dipartimenti o capacità di business diversi possono avere modelli di dominio diversi e/o linguaggi diversi. Sottodomini diversi potrebbero:

- usare lo stesso nome per entità concettuali diverse;
- usare nomi diversi per indicare la stessa entità concettuale;
- vedere la stessa entità concettuale da prospettive diverse;
- vedere relazioni diverse tra le stesse entità.

I sottodomini emergono dal lavoro fatto fin qui (Figura 14): dagli obiettivi generali si passa ai value stream e ai support stream, e da questi alle attività e ai subgoal, da cui si ricavano attori e stakeholder, regole e policy, entità e linguaggio.

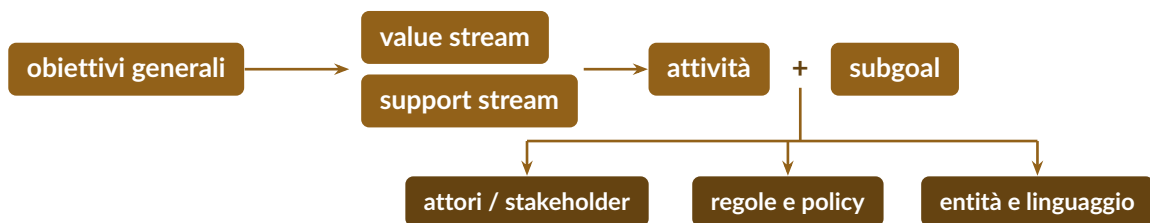


Figura 14: Dagli obiettivi generali agli elementi che fanno emergere i sottodomini

Dopo aver individuato i sottodomini del nostro dominio, può avere senso suddividerli in tre macrocategorie: **core**, **supporting** e **generic**. La distinzione riguarda due assi: la **complessità** del sottodominio e il **vantaggio competitivo** (Figura 15).

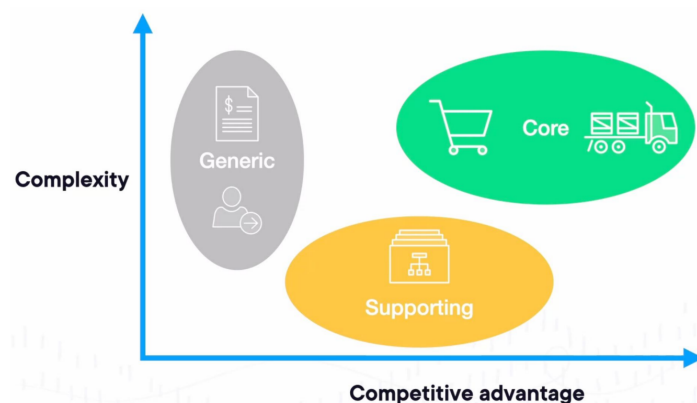


Figura 15: Macrocategorie di sottodominio

Core Funzioni che danno valore al business: costituiscono il valore aggiunto che consente al business di fare affari. Ci aspettiamo che siano complesse, e sono le funzioni per cui siamo disposti a investire.

Supporting Funzioni che supportano l'attività. Sono sottodomini abbastanza specifici dell'attività, ma non hanno né una grossa complessità né costituiscono un grosso vantaggio competitivo: ci devono essere, ma non sono ciò che ci preoccupa.

Generic Funzioni uguali in moltissimi business: possono anche essere molto complesse (es. la contabilità), ma sono uguali a quelle degli altri. Per questi sottodomini si possono acquistare prodotti già esistenti.

2.6.2 Bounded context

Bounded context (BC)

Confine dentro cui un certo modello di dominio viene applicato in modo coerente e consistente. Stabilisce con chiarezza a chi appartiene il modello (*ownership*): all'interno del confine la terminologia, le regole di business e la logica di funzionamento sono coerenti e prive di ambiguità.

— Eric Evans, il primo autore a introdurre il DDD

I principi chiave sono:

- **incapsulamento**: cose molto collegate tra loro stanno nello stesso bounded context;
- **forte coesione** all'interno del bounded context;
- **basso accoppiamento** (*low coupling*): bounded context diversi non hanno troppi collegamenti tra loro;
- **interazione chiara** tra bounded context diversi: deve essere evidente quali sono i punti in cui le informazioni o le entità viaggiano tra un bounded context e l'altro, e le ragioni per cui succede.

Il ragionamento è simile a quello che si fa quando si identificano gli oggetti e le classi rilevanti all'interno di un dominio. Qui lo si fa a un livello più alto: si identificano domini e funzioni di business che siano in qualche modo indipendenti l'uno dall'altro.

2.6.3 Dai sottodomini ai bounded context

I sottodomini sono un punto di partenza. Un bounded context è qualcosa che offre un **insieme di servizi coeso**: ragionare in termini di servizi (concettuali) può dare ulteriori indicazioni su come passare dagli uni agli altri.

One-2-one Un sottodominio, un bounded context.

Many-2-one Un solo bounded context per due o più sottodomini:

- quando alcune operazioni, o la maggior parte, richiedono forti interazioni tra funzioni di sottodomini diversi;
- quando i modelli dei dati si sovrappongono troppo;
- quando le scelte tecnologiche, o i loro limiti, impediscono o ostacolano seriamente il disaccoppiamento: ad esempio quando è già presente un software che fa queste cose, ed è uno solo.

One-2-many Un sottodominio diviso in due o più bounded context:

- quando posso dividere le funzioni in blocchi largamente indipendenti, oppure stratificati (uno strato che sta dietro e offre dei servizi allo strato che sta davanti): c'è cioè una chiara suddivisione tra funzioni che vengono prima e producono qualcosa, e funzioni che lo usano;
- quando le esigenze tecnologiche ci permettono, o ci obbligano, a dividere (es. front-end e back-end).

2.6.4 Interazioni tra bounded context

Un modo per capire se abbiamo collegato bene i bounded context è studiare le **dinamiche di potere** che ci sono tra di essi.

Ricordiamo che un bounded context non è un software: è un contesto che include il team di sviluppo, i processi di sviluppo e anche il software prodotto. Le dinamiche di potere fra bounded context sono quindi le dinamiche di potere tra i team che sviluppano i diversi sistemi. I pattern sono quattro.

Separate Ways I due BC non hanno niente a che fare l'uno con l'altro: i team che li sviluppano possono procedere in modo completamente indipendente, perché tra di loro non c'è nessuna interazione.

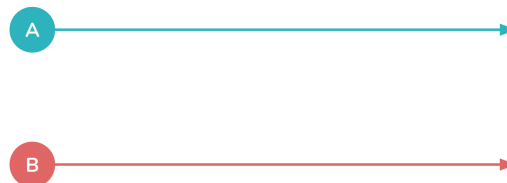


Figura 16: Separate Ways

Partnership Due bounded context che lavorano fianco a fianco in modo continuativo e hanno obiettivi allineati. I due software si scambieranno spesso delle informazioni, e i team di sviluppo dovranno spesso trovare accordi comuni su alcuni punti dello sviluppo.

Qui si crea un **alto accoppiamento**. Bisogna quindi riflettere se tenere due microservizi separati, oppure fare un unico microservizio con due moduli separati all'interno, che per parlarsi non hanno bisogno di una comunicazione di rete.

Anche quando emerge un accoppiamento molto alto, ci sono casi in cui è vantaggioso mantenere due microservizi separati:

- se, a fronte della necessità di comunicare, le entità su cui lavorano sono abbastanza diverse;
- per motivi tecnologici, ad esempio se per qualche ragione i due devono girare su due piattaforme diverse (in questo caso sono due per forza).

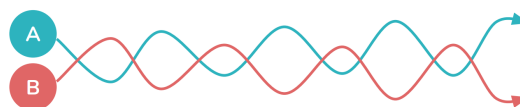


Figura 17: Partnership

Customer / Supplier È il caso più comune: un servizio (il *supplier*) fornisce funzionalità all'altro (il *customer*). Magari non fa solo quello, ma la relazione tra i due è presente.

Per i team di sviluppo, in generale, questo significa che i due team si dovranno mettere d'accordo su un'interfaccia comune: il customer potrà esprimere le sue esigenze al supplier, che ne terrà conto.

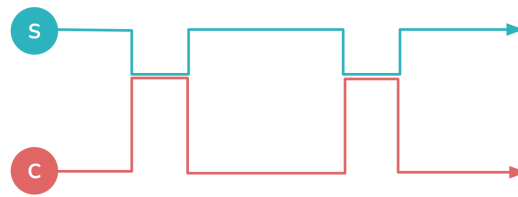


Figura 18: Customer / Supplier

Conformist È simile al Customer / Supplier, ma in questo caso il servizio che offre le funzionalità non è modificabile: il team che sviluppa il customer non può fare richieste perché l'altro gli venga incontro.

Succede spesso quando uso software di terze parti: sono io che lo uso a dovermi adeguare a ciò che offre. In questo caso può presentarsi un problema di ubiquitous language, perché il software esterno userà il suo linguaggio.

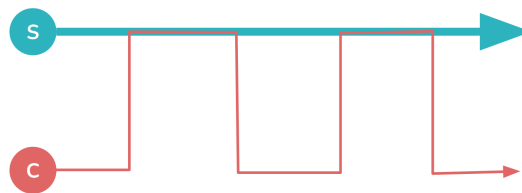


Figura 19: Conformist

2.6.5 Shared Kernel e Anti-Corruption Layer

I quattro pattern appena visti danno per scontato che i due BC abbiano modelli di dominio indipendenti, ciascuno con il proprio linguaggio. Ci sono però due casi che modificano questo quadro: il primo si lega alla Partnership, il secondo al Conformist.

Shared Kernel Ci sono casi in cui vale la pena di **condividere una porzione** del modello di dominio, possibilmente piccola e comunque ben definita (Figura 20).

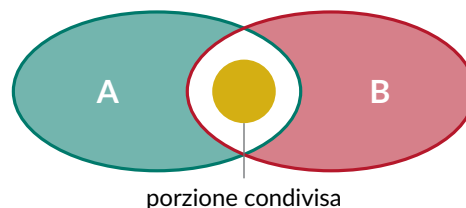


Figura 20: Shared Kernel: i BC A e B condividono una porzione del modello di dominio

- Condividere crea un **alto accoppiamento** tra i BC coinvolti: ci devono essere buone ragioni per progettare così i modelli di dominio, e avere una comprensione identica dei concetti chiave nei due BC deve essere una vera necessità.
- In molti casi conviene invece replicare quella porzione nei due BC, soprattutto se è un pacchetto molto piccolo, per non rischiare di fare confusione.
- Quando c'è uno shared kernel si crea una **partnership**: inevitabilmente i team dovranno comunicare spesso e portare avanti delle cose insieme, e servono fiducia e collaborazione.
- L'ideale è che la parte condivisa **evolva lentamente**, così l'impatto sui due BC è minore.

Anti-Corruption Layer Nel caso del Conformist può essere utile inserire nel customer un **Anti-Corruption Layer** (ACL): uno strato, all'interno del modello di dominio del BC, che lo protegge dai modelli esterni **traducendo** ciò che arriva da fuori nell'ubiquitous language locale (Figura 21).

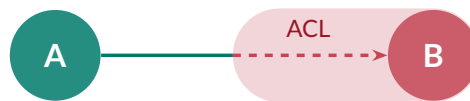


Figura 21: Anti-Corruption Layer: ciò che arriva da A viene tradotto prima di entrare nel modello di B

- Evita che il BC venga «inquinato» dal linguaggio di un altro BC, o di un fornitore esterno.
- È una difesa contro le relazioni di tipo Conformist: il team del BC non deve progettare «intorno» alle scelte di qualcun altro.
- Se il linguaggio esterno cambia, va aggiornato solo l'ACL, non l'intero modello.
- È una scelta costosa in termini di complessità e di impegno: deve valerne la pena.

Esempio

Mi arriva un oggetto JSON da un servizio esterno che, per nomi delle proprietà e per struttura, non corrisponde per niente a come ho rappresentato quegli oggetti al mio interno. Metto uno strato di traduzione: in questo modo, se un giorno dovrò cambiare il servizio esterno, dovrò modificare solo lo strato di traduzione e non il mio sistema.

2.6.6 Front-end

Il front-end di un'applicazione erogata tramite web ha esigenze tecnologiche diverse, quindi inevitabilmente finisce per essere separato dal back-end.

Dalla prospettiva del DDD e dei BC, esistono tipicamente tre tipi di front-end.

1. **Front-end specifico per uno specifico BC**, o per un sottoinsieme delle funzioni di business offerte da un BC (es. un front-end solo per i cuochi). L'utente interagisce con un solo BC e ne condivide l'ubiquitous language: il front-end è solo una questione di scelte tecnologiche.
2. **«Portale» o «federazione», multi-BC**. Ci sono tanti BC che offrono ciascuno le proprie funzioni di business, ma voglio creare un portale che ci metta sopra una stessa UI/UX: gli utenti devono percepire un contenitore uniforme.

Con un approccio DDD questo può essere un po' complesso: se i BC hanno ubiquitous language diversi, può voler dire che coprono aree diverse. Però, se sto pensando di fare un portale, mi aspetto che quelle funzioni siano offerte alla stessa tipologia di pubblico: ha quindi senso cercare, almeno a un livello superficiale, un'uniformità di linguaggio nella presentazione verso gli utenti finali.

3. **Orchestratore, cross-BC**. Il front-end orchestra un processo dell'utente che interagisce con diversi BC. In questo caso, tipicamente, il front-end ha un BC tutto suo, il cui dominio è il percorso dell'utente (*user journey*) o il workflow.

2.6.7 Mockup

Una pratica utile per fare l'analisi del front-end è quella dei mockup.

Mock UI

Serie di immagini che forniscono un minimo di navigabilità e permettono quindi di intuire come potrebbe essere la UX: mostrano come l'utente potrebbe interagire con il sistema per raggiungere i suoi obiettivi.

I mockup rendono visibile il modello di dominio, così che lo si possa discutere e validare. Sono semplici screenshot, eventualmente navigabili, senza alcuna implementazione; uno strumento pratico per costruirli, con licenza educational gratuita, è **Figma**.

I primi mockup:

- seguono le user stories selezionate, e permettono di provarle meglio;
- abbozzano come avviene l'interazione;
- permettono di **testare i bounded context**, perché vedo che linguaggio mi viene da usare nella UI:
 - se un workflow attraversa più BC, è un caso di orchestrazione e va bene così;
 - se concetti di BC diversi finiscono sempre «incollati» insieme, c'è un problema di partizionamento;
- fanno emergere quale **tipo di front-end** scegliere: applicazione per un singolo BC, portale federato oppure orchestratore.

2.7 Tactical DDD

Il Tactical DDD si occupa di **implementare** il modello di un (sotto)dominio all'interno di un bounded context. Va quindi verso l'implementazione, ma è un'implementazione **agnostica** rispetto al linguaggio di programmazione e alla tecnologia:

- ragioniamo in termini di classi, attributi e metodi, ma **non scriviamo codice** (potremmo anche non usare un linguaggio a oggetti);
- l'idea è ancora concettuale: definiamo solo le interfacce delle classi, cioè quello che fanno, e non le definizioni dei metodi, cioè come lo fanno.

Quello che otteniamo è un **class diagram UML** con degli specifici **stereotipi**: uno stereotipo è l'etichetta che sta in alto, tra parentesi angolari. Sono gli elementi tattici introdotti dal DDD, e rappresentano il ruolo che le classi giocano dentro quel BC e l'una rispetto all'altra.

2.7.1 Entity

Entity

Un concetto del dominio definito dalla sua **identità**, e non dai suoi attributi: rappresenta un «qualcosa» che persiste e che può cambiare nel tempo, mantenendo la sua identità.

Le entità sono i concetti rilevanti del modello di dominio. In un software, sono ciò che ha bisogno di un **id** per essere identificato: tutti gli altri attributi potrebbero cambiare, ma l'entità resta lei.

Nel DDD si distingue tra implementazioni **anemiche** e **ricche** delle entità:

- **anemica**: contiene solo i dati dell'entità;

- **ricca**: include anche il comportamento (Figura 22).

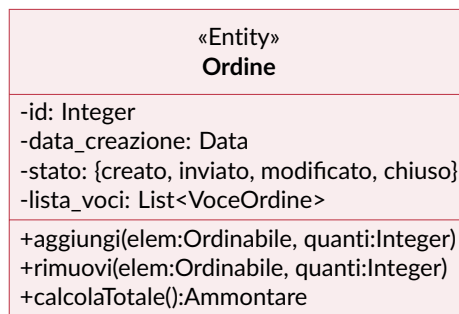


Figura 22: Esempio di entità ricca

Il DDD raccomanda **entità ricche** per lo strato di business logic.

Le entità anemiche compaiono nel software quando un'entità deve attraversare i **confini** del bounded context: se devo comunicare delle informazioni su un'entità a un altro BC, la posso tradurre in una versione anemica, perché nell'altro BC le sue funzioni potrebbero non avere senso. Nel codice queste versioni diventano i **DTO**.

2.7.2 Value Object

Value Object

Un concetto del dominio **immutabile**, cioè definito dai suoi **attributi**.

Gli attributi di un value object vengono impostati nel momento in cui l'oggetto nasce, e non possono cambiare: se un attributo cambia, in realtà si tratta di un altro oggetto (Figura 23).



Figura 23: Esempio di value object

Questi oggetti possono essere trattati in maniera più semplice: secondo il DDD è utile riuscire a capire presto quali sono.

2.7.3 Aggregate

Aggregate

Un insieme di oggetti che devono essere mantenuti consistenti tra loro, e che per questo vengono trattati come un'**unità**.

Una delle entità viene eletta **radice dell'aggregato** (*aggregate root*):

- le entità esterne possono interagire con l'aggregato **solo attraverso la radice**: possono sapere che le entità interne esistono, ma non possono tenerne un riferimento né modificarle;
- la radice è responsabile di mantenere e garantire la **consistenza** all'interno dell'aggregato.

Per scegliere la radice ci si può chiedere:

- a quale oggetto fanno riferimento le altre parti del dominio, o gli altri BC?
- quale oggetto è responsabile di far rispettare le regole e la consistenza sugli altri?

La radice è il «capitano» dell'aggregato. Se all'interno di un aggregato ci sono due candidati in competizione e non riesco a decidermi, forse quello non è un aggregato.

Tipicamente l'aggregato ha dentro una **relazione di composizione** (qualcosa che ha dei pezzi, delle parti), oppure più relazioni di composizione (qualcosa che ha delle parti, che a loro volta hanno delle parti), come nell'esempio in Figura 24.

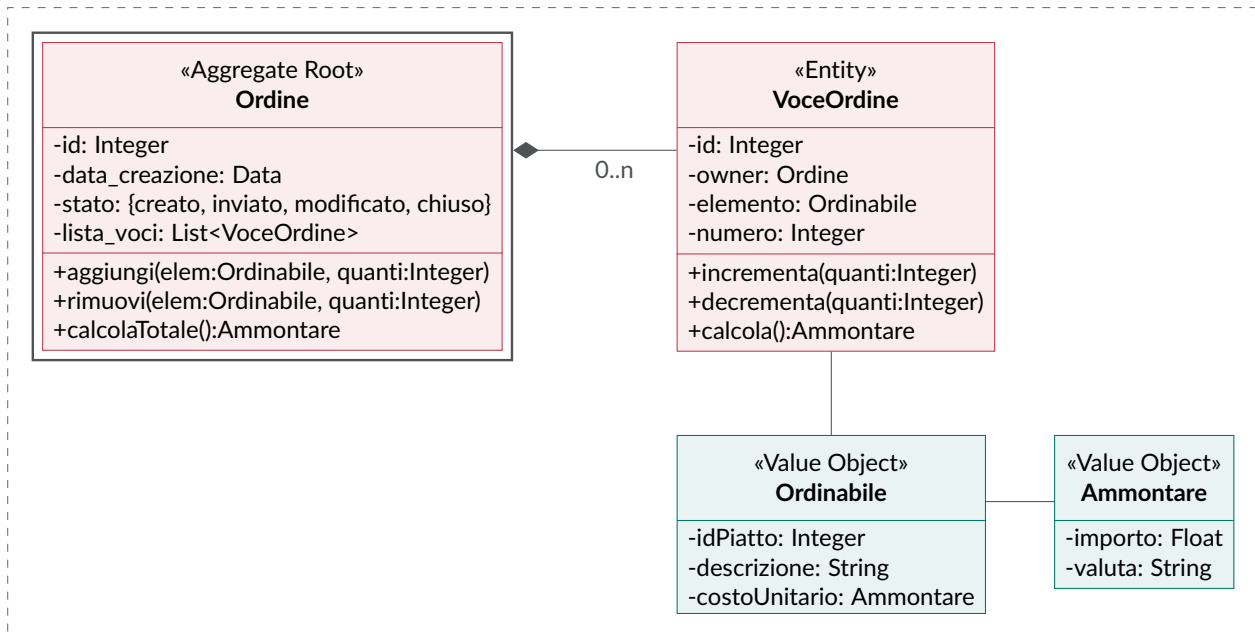


Figura 24: Esempio di aggregato: il confine tratteggiato racchiude gli oggetti dell'aggregato, la doppia cornice ne indica la radice

2.7.4 Factory

Quando un insieme di oggetti (quasi sempre all'interno di un aggregato) deve essere creato in modo coerente, rispettando regole o invarianti complessi, non posso demandare a ogni singolo oggetto la possibilità di creare se stesso: la creazione deve avvenire di concerto con altri oggetti esistenti. In questi casi posso mettere nel bounded context un **factory object**, a cui delegare la creazione.

Inserire una factory significa rendere esplicito che gli oggetti di quel gruppo non possono crearsi da soli, ma hanno bisogno di **coordinamento**: solo una terza parte che li vede tutti può effettuare quell'operazione garantendo la consistenza.

Spesso è l'aggregate root a fare da factory per i suoi componenti: la factory non deve per forza essere un oggetto separato, ma può esserlo. Ci sono però due cose da tenere presenti:

- le factory riguardano gruppi generici di oggetti, non solo gli aggregati;
- a volte c'è della **circolarità**: un oggetto non può avere un comportamento finché non esiste, quindi non può garantire la propria consistenza finché non è stato creato. In programmazione questo si risolve spesso con i metodi statici, oppure separando la creazione dall'inizializzazione dell'oggetto, come fa Spring.

2.7.5 Repository

Repository

Uno strato di astrazione sui dati (*Data Abstraction Layer*) che permette di **disaccoppiare** l'implementazione della business logic del dominio dalla sua rappresentazione persistente.

Un repository:

- salva e recupera gli aggregati, rendendo la business logic indipendente da *come* avviene la persistenza (su file? in un database relazionale? in un database NoSQL? tramite un servizio esterno?);
- permette di salvare e recuperare gli aggregati come un tutt'uno.

Il repository compare quindi nel modello quando, già a livello concettuale, c'è l'esigenza di separare due cose: la rappresentazione di un oggetto che uso per lavorarci, e la versione che verrà resa persistente. In un bounded context è l'elemento in grado di tradurre l'oggetto dall'una all'altra, cioè in un certo formato.

Non succede così di frequente che questa separazione si crei a livello logico. Un caso è quello in cui la persistenza è gestita da qualcuno di esterno: devo allora creare una sorta di anti-corruption layer, che trasformi l'oggetto secondo le esigenze di quella rappresentazione esterna.

Non è il repository JPA di Spring

Il principio è simile, ma **non sono la stessa cosa**.

- Con un repository JPA (*Java Persistence API*) ho già fatto la scelta di un database relazionale.
- Il repository JPA non traduce un oggetto concettuale in una diversa rappresentazione adatta al database: quando ho un repository JPA ho già una rappresentazione adatta al database (la `@Entity` di Spring), e ciò che fa JPA è l'azione fisica di scrivere nel database, sempre relazionale.
- Dal punto di vista del DDD, quindi, un repository JPA è un dettaglio implementativo: sta a un livello più basso di un repository DDD.

2.7.6 Service

Service

In un bounded context, è l'elemento che rappresenta le operazioni di business logic che non possono essere ascritte a una singola entità o a una singola aggregate root, perché richiedono la **cooperazione di più entità**.

Quando non ha senso dare quelle funzionalità a una delle entità, si usa un service. Il service non contiene dati e non possiede informazioni: il suo ruolo è comporre le funzionalità delle altre entità in una funzionalità più complessa. Fa quindi da **orchestratore**, ed è tipicamente **stateless**.

Non è il @Service di Spring

Il principio è simile, ma **non sono la stessa cosa**. Spesso i service di Spring fanno solo da ponte tra i controller e le entità: sono più che altro uno strato intermedio, che astrae i dettagli dell'imple-

mentazione delle entità nel database e offre delle funzioni ai controller. I controller le invocano per ottenere gli oggetti da mandare in risposta, oppure per modificare le entità nel database.

In sintesi

- Non tutti i sistemi beneficiano dei microservizi: il **monolite** va bene finché è piccolo (un solo team DevOps, 5–10 persone); quando cresce troppo si arriva all'**inferno monolitico**.
- Per la **legge di Conway** i sistemi copiano le strutture comunicative di chi li progetta: con la manovra inversa si ristruttura l'organizzazione a immagine dell'architettura.
- Le **sfide** dei microservizi: complessità, ristrutturazione organizzativa, prestazioni di rete, indisponibilità dei servizi, transazioni (CAP theorem) e sicurezza servizio per servizio.
- Il **DDD** mette al centro il **modello di dominio**, costruito insieme all'**ubiquitous language** in modo iterativo; gli strumenti sono event storming, value stream, user stories e CRC cards.
- Lo **Strategic DDD** divide il dominio in **sottodomini** (core, supporting, generic) e poi in **bounded context**, che interagiscono secondo quattro pattern: Separate Ways, Partnership, Customer / Supplier e Conformist, a cui si aggiungono Shared Kernel e Anti-Corruption Layer.
- Il **Tactical DDD** modella l'interno di un bounded context con un class diagram UML e gli stereotipi **Entity**, **Value Object**, **Aggregate**, **Factory**, **Repository** e **Service**.

3 REST

3.1 Operazioni remote

Operazione

Un'**operazione** è un contratto tra qualcuno che chiama l'operazione (il *chiamante*) e qualcuno che viene chiamato (il *chiamato*), che si presuppone faccia qualcosa in seguito a questa chiamata. Le caratteristiche che identificano un'operazione sono:

- il **nome**, che identifica l'operazione;
- gli **input** necessari per completarla;
- gli **output** attesi, cioè i suoi risultati;
- gli **errori** che potrebbe incontrare, e segnalare, durante l'esecuzione.

In generale una **API** (*Application Programming Interface*), a prescindere dal contesto in cui viene usato questo termine, deve esprimere quali sono le operazioni a cui quel particolare servizio, o soggetto, risponde.

Esempio. L'API di una classe Java è l'elenco dei metodi di quella classe. L'API di un back-end è un elenco di URL (*endpoint*) che si possono usare per rivolgere richieste a quel back-end.

Nei linguaggi di programmazione le operazioni sono solitamente tradotte come **procedure**, **funzioni** o **metodi**. Ad esempio:

```
creaOrdine(idTavolo: long) → Ordine
```

3.1.1 Chiamata locale e chiamata remota

Nella **chiamata locale** è facile: il chiamante e il chiamato stanno nello stesso processo, nello stesso spazio di memoria, quindi passarsi queste informazioni non è tanto difficile.

Nel momento in cui andiamo a eseguire le operazioni **in remoto** le cose cambiano un po', perché in mezzo tra il chiamante e il chiamato c'è la rete (Figura 25). Dobbiamo trovare un modo, attraverso la rete, di:

- **identificare l'operazione** che il chiamante vuole invocare;
- **mandare gli input** sulla rete;
- far viaggiare **gli output, o gli errori**, nella direzione opposta.

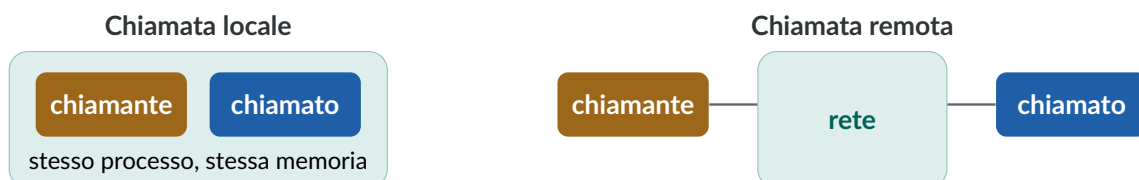


Figura 25: Chiamata locale e chiamata remota

Questo provoca delle complicazioni. Le principali sono due.

Serializzazione dei dati È la complicazione principale. Finché sono in locale posso dare per scontato che tutte le parti del mio applicativo condividano la rappresentazione dei dati (es. quanti byte usare per un tipo, come viene rappresentato l'oggetto in memoria, il puntatore o il riferimento all'oggetto in memoria). In remoto, invece, dobbiamo essere in grado di mandare sulla rete tutto quello che serve: bisogna tradurre i dati di input in un modo che sia comprensibile, e traducibile, per chi li riceve.

Esempio. In JavaScript, l'operazione con cui traduciamo un oggetto JavaScript in JSON si chiama **serializzazione**; quella con cui traduciamo un JSON in un oggetto JavaScript si chiama **deserializzazione**.

Nuovi errori La rete introduce nuovi errori: potrebbe andare tutto bene dal punto di vista di chi esegue l'operazione, ma se c'è un errore di rete il chiamante potrebbe non ricevere la risposta, o riceverla danneggiata.

3.2 Operazioni remote su HTTP

HTTP mette a disposizione gli strumenti per fare tutte e tre le cose.

Che cosa serve	Che cosa si può usare
Identificare l'operazione	Il METHOD di HTTP (POST, GET, PUT, DELETE, ecc.) e la ROUTE (il path); per configurare l'operazione, gli HEADERS .
Mandare gli input in rete	Stabilito che li traduciamo in qualche modo in testo: i PATH PARAMS (pezzi della route che vengono interpretati come input), il BODY della richiesta (con POST, PUT o PATCH) oppure la QUERYSTRING (il pezzo di URL che viene dopo il ?).
Ottenere indietro i risultati, o gli errori	Lo STATUS CODE di HTTP e il BODY della risposta.

Quando definiamo una API remota in termini di operazioni, con i loro input e i loro output, stiamo in realtà dicendo che questa API segue lo **stile RPC**.

RPC (Remote Procedure Call)

Paradigma di comunicazione, tra i primi nel software distribuito, in cui l'idea è permettere a chi scrive il codice di chiamare delle operazioni remote **come se fossero locali**.

Il vero RPC è un approccio in cui si cerca di nascondere al programmatore il fatto che l'operazione sia effettivamente in remoto: il chiamante usa una chiamata di funzione identica a quella che userebbe se l'operazione fosse in locale, e non sa (né ha bisogno di sapere) se è locale o remota. Per rendere possibile tutto questo, nel vero RPC bisogna prevedere tutto un macchinario (un framework) dietro le quinte.

Noi utilizzeremo un **RPC style**: l'idea, cioè, di usare HTTP per esprimere un'operazione, ragionando in termini di operazioni, input e output.

Una API in stile RPC *pensa* come RPC, ma non si *comporta* come RPC, perché di fatto non abilita la trasparenza delle chiamate: quando scrivo la mia chiamata me ne accorgo, che è una chiamata di rete e non una procedura interna.

3.2.1 Esempio di API su HTTP in stile RPC

L'operazione `creaOrdine` vista prima si può esprimere su HTTP in modi diversi.

1. La route è totalmente ignara dell'operazione: tutte le informazioni relative all'operazione sono nel body.

```
POST /api { "op": "creaOrdine", "idTavolo": 3 }
```

2. La route esprime il nome dell'operazione, e il body ne esprime l'input.

```
POST /creaOrdine { "idTavolo": 3 }
```

3. Faccio una GET invece di una POST, per non avere il body, e metto il parametro nella querystring.

```
GET /creaOrdine?idTavolo=3
```

Una volta questa forma era molto usata nel web. È particolarmente brutta perché **viola le regole di HTTP**, secondo cui la GET deve essere un'operazione *safe*, che non modifica lo stato del sistema a cui mi sto rivolgendo.

Per quanto riguarda l'output: se va tutto bene avrò una risposta HTTP con uno **status di ok**, e nel body della risposta ci sarà il risultato dell'operazione (qui, la rappresentazione dell'ordine creato); se invece qualcosa è andato storto, avrò uno **status di errore**.

3.3 Representational State Transfer (REST)

L'RPC style ogni tanto è necessario: ci sono alcuni tipi di API che non possono che essere espresse in questo stile. Uno dei suoi problemi, però, è che non va tanto d'accordo con il modo in cui è costruito HTTP, e soprattutto con il modo in cui sono costruite le **URL** (*Uniform Resource Locator*): le URL nascono per identificare delle **risorse**, non per esprimere delle operazioni.

Questo ha fatto emergere l'idea di pensare a uno stile architetturale per le API che si basasse non sul concetto di operazione, ma sul concetto di risorsa. È un'idea che va particolarmente d'accordo con il mondo di quegli applicativi, tendenzialmente gestionali, che sono il 90%: quelli in cui le operazioni che vengono fatte attraverso l'applicativo sono operazioni **CRUD** (*Create, Read, Update, Delete*) sul database.

REST

REST è uno **stile architetturale** per progettare API di rete: specifica la forma e la semantica della comunicazione tra un fornitore (il server) e un consumatore (il client) del servizio.

Sapendo che la maggior parte delle applicazioni manipola risorse, nasce l'idea di dire che il servizio non esegue operazioni, ma **gestisce risorse**.

- Il fornitore del servizio gestisce delle **risorse**: sono concettuali, ad esempio le Entity del DDD.
- Chi consuma il servizio può manipolare una risorsa scambiando con il fornitore delle **rappresentazioni** di quella risorsa.
- Le operazioni avvengono tramite richieste **stateless**, con un'interfaccia uniforme: i metodi HTTP, la rappresentazione tramite URI e degli status code con un significato preciso.

Richieste stateless

Una richiesta è **stateless** quando è indipendente dalle precedenti. O meglio: ogni richiesta può modificare lo stato interno del servizio, e questo altererà il comportamento delle richieste successive (es. se faccio una DELETE e cancello una risorsa, con una GET non la troverò più); ma tendenzialmente il fornitore del servizio **non ricorda nulla delle chiamate precedenti**.

REST non è necessariamente collegato a HTTP, ma di fatto è diventato lo stile architetturale con cui si rappresentano le API sopra HTTP, perché HTTP si presta naturalmente: usa le URL, che sono indirizzi di risorse, ed è stateless.

È chiaro che, perché il fornitore e il cliente del servizio possano parlarsi, bisogna essersi messi d'accordo sulla specifica delle API, e che entrambi devono rispettarla; ma questo vale anche per l'altro stile.

3.3.1 Risorse REST

Le **URI** (*Uniform Resource Identifier*) degli endpoint, cioè le route, devono esprimere **nomi** di risorse, non azioni. Ci accorgiamo subito se un'API non è REST, perché nelle route troviamo dei verbi come *crea*, *modifica*, *sposta*, ecc.

Questa impostazione si sposa bene con il DDD:

- i **segmenti** del path rappresentano le **entità** (il tipo, seguito dall'id);
- i segmenti **annidati** rappresentano oggetti **aggregati** o contenuti.

Esempio di risorse REST

/order	tutti gli ordini
/order/54	l'ordine con id = 54
/menu/3/items	tutti gli item del menu con id = 3
/menu/3/items/15	l'item con id = 15 del menu con id = 3

3.3.2 Verbi REST

I verbi esprimono l'azione compiuta su una risorsa, e corrispondono ai metodi HTTP. Ciascuno è caratterizzato da due proprietà: se ha **side effects**, cioè se modifica lo stato, e se è idempotente.

Idempotenza

Un'operazione è **idempotente** se, ripetendo due volte la stessa azione, ottengo lo stesso risultato che avrei ottenuto facendola una volta sola.

GET Viene usata per **ottenere** la rappresentazione della risorsa specificata: è solo in lettura.

- Non ha side effects (non modifica lo stato) ed è **idempotente**.
- Non ha il corpo della richiesta; ci si aspetta che il body della risposta contenga la rappresentazione della risorsa.
- La query string può essere usata per esprimere **filtri** o **proiezioni**, per limitare i dati restituiti.

Esempio

GET /order	mi aspetto di ottenere tutti gli ordini
GET /order/54	mi aspetto di ottenere il singolo ordine con id = 54
GET /order?status=confirmed	ottengo tutti gli ordini con stato confermato

POST Viene usata unicamente per **creare** una nuova risorsa.

- Ha side effects e **non è idempotente**.
- Il corpo della richiesta deve contenere la rappresentazione della **nuova** risorsa da creare.
- La query string può essere usata per esprimere dei modificatori o delle opzioni per la richiesta.

Esempio

POST /order	con un body che rappresenta il nuovo ordine
POST /order?cache=true	lo stesso, con un'opzione nella query string

Pragmatic REST

Nella specifica REST originale il corpo della richiesta poteva essere un XML oppure un JSON. In quello che viene chiamato **pragmatic REST**, cioè il REST che viene realmente utilizzato (in particolare nelle architetture a microservizi), si usa solo JSON: lo si preferisce perché è meno verboso e più sintetico.

PUT **Sostituisce** una risorsa esistente con una nuova rappresentazione: va ad agire su qualcosa che ha già un id.

- Ha side effects, ma è **idempotente**.
- Il corpo della richiesta contiene la rappresentazione **aggiornata** della risorsa.
- La risposta dà indietro la nuova rappresentazione dell'entità in questione.
- La query string si usa come per la POST, per modificatori e opzioni.

Esempio

PUT /order/42	con un body che rappresenta l'ordine aggiornato
PUT /order/42?mirror=all	lo stesso, con un'opzione nella query string

DELETE **Cancella** una risorsa esistente.

- Ha side effects ed è **idempotente**: se cancello una risorsa e poi provo a ricancellarla, quella risorsa non c'è già più.
- Non ha il corpo della richiesta.
- La query string si usa come per la POST, per modificatori e opzioni.

Esempio

DELETE /order/42/items cancella tutti gli item dell'ordine con id = 42
 DELETE /order/42/items/15 cancella l'item con id = 15 dell'ordine con id = 42

PATCH Aggiorna solo una parte di una risorsa: non la rimpiazza interamente, ma va a cambiarne solo alcuni campi.

- Ha side effects, e può essere **idempotente oppure no**, a seconda del tipo di aggiornamento («imposta il valore a 5» contro «incrementa il valore»).
- Il corpo della richiesta contiene o una rappresentazione **parziale** della risorsa (solo i campi che devono essere modificati), oppure un'indicazione di **come** la risorsa deve essere modificata.
- La query string si usa come per la POST, per modificatori e opzioni.

Esempio

La stessa richiesta PATCH /order/42/items/15, con due body diversi:

{ "price": 12 } è idempotente
 { "number": "+1" } non è idempotente, perché non ottengo sempre lo stesso risultato

La Tabella 3 riassume le caratteristiche dei cinque verbi.

Verbo	Azione	Side effects	Idempotente	Corpo della richiesta
GET	ottiene una risorsa	no	sì	nessuno
POST	crea una nuova risorsa	sì	no	rappresentazione della nuova risorsa
PUT	sostituisce una risorsa	sì	sì	rappresentazione aggiornata della risorsa
DELETE	cancella una risorsa	sì	sì	nessuno
PATCH	aggiorna parte di una risorsa	sì	dipende	rappresentazione parziale, oppure modifiche da applicare

Tabella 3: I verbi REST a confronto

3.3.3 Status code delle risposte

REST dà anche una specifica formale di come devono essere usati gli status code nelle risposte: a ciascuno associa un significato preciso, che va rispettato (e usato!) se si vuole essere conformi a REST.

Codice	Nome	Quando si usa
2XX – Success		
200	OK	Successo generico.
201	Created	Una POST che crea con successo una nuova risorsa.
204	No Content	L'operazione ha successo, ma non viene restituito un body: per esempio per DELETE o PUT.

Codice	Nome	Quando si usa
3XX – Redirection		
303	See Other	Può essere usata dopo la POST (non è obbligatoria): reindirizza alla GET della risorsa appena creata.
4XX – Client error		
400	Bad Request	Richiesta mal costruita.
401	Unauthorized	Credenziali invalide o mancanti.
403	Forbidden	L'utente è autenticato, ma il suo ruolo non gli permette di fare quella cosa.
404	Not Found	La risorsa non esiste.
409	Conflict	L'operazione che ho chiesto di fare confligge con lo stato del sistema.
422	Unprocessable Entity	Viola le regole semantiche del dominio.
429	Too Many Requests	Limite di richieste superato.
5XX – Server error: errori dovuti a mancanze che non sono problemi logici, ma problemi sostanziali		
500	Internal Server Error	Eccezione generica.
502	Bad Gateway	Fallimento dovuto al fatto che il nostro servizio sta cercando di contattare un altro servizio che non funziona (per i proxy).
503	Service Unavailable	Servizio non attualmente disponibile, ma ci si aspetta che torni a esserlo dopo un po' (riprovare più tardi).
504	Gateway Timeout	Qualcun altro a cui mi sto rivolgendo va in timeout (per i proxy).

Tabella 4: Gli status code nelle risposte REST

3.4 REST vs RPC

REST e RPC hanno due filosofie completamente diverse.

	REST	RPC
Endpoint	Identifica la risorsa ; è il verbo a dire che cosa dobbiamo farne.	Esprime l' operazione da eseguire.
Body	Contiene la rappresentazione della risorsa.	Rappresenta i parametri (nella richiesta) o il valore restituito (nella risposta).

Volendo essere **pragmatici** (*pragmatic REST*), a volte è necessario usare gli endpoint per esprimere delle operazioni, anche se abbiamo i verbi:

- a volte i METHOD non sono abbastanza per far capire la natura dell'operazione, perché la sua semantica è troppo fine per limitarsi ai verbi: dobbiamo allora inserire un'ulteriore specifica, per far capire che cosa vogliamo fare con quella risorsa;
- a volte non c'è proprio l'idea di una risorsa: l'operazione non corrisponde a una risorsa «esistente» sul server.

In questi casi possiamo cercare di essere **il più REST possibile**:

- se è comunque coinvolta una risorsa, ma dobbiamo aggiungere un verbo, cerchiamo di mettere il verbo **alla fine**, dopo aver indicato qual è la risorsa su cui vogliamo operare;

- cerchiamo di rispettare la **caratterizzazione dei verbi**: non usare GET se ci sono dei side effects; usare PUT se l'operazione modifica lo stato ed è idempotente, POST se lo modifica e non è idempotente, ecc.

Per l'**RPC puro** bisogna invece guardare ad altri protocolli: qui entra in gioco **gRPC**, che verrà approfondito più avanti nel corso.

3.5 Dove si usa REST nelle MSOA

REST si usa per la **comunicazione tra front-end e back-end** (Figura 26).

Di solito si mette un *reverse proxy* che fa da interfaccia con il mondo esterno (nella figura, la *front door*): il front-end parla con il reverse proxy, e questo, in base alla composizione della URL, capisce a quale servizio indirizzare la richiesta. Il front-end parla sempre con un unico indirizzo, e non vede tutti i microservizi.

REST può essere usato anche per la **comunicazione tra i servizi**, quando devono parlarsi tra di loro: è una delle opzioni possibili.

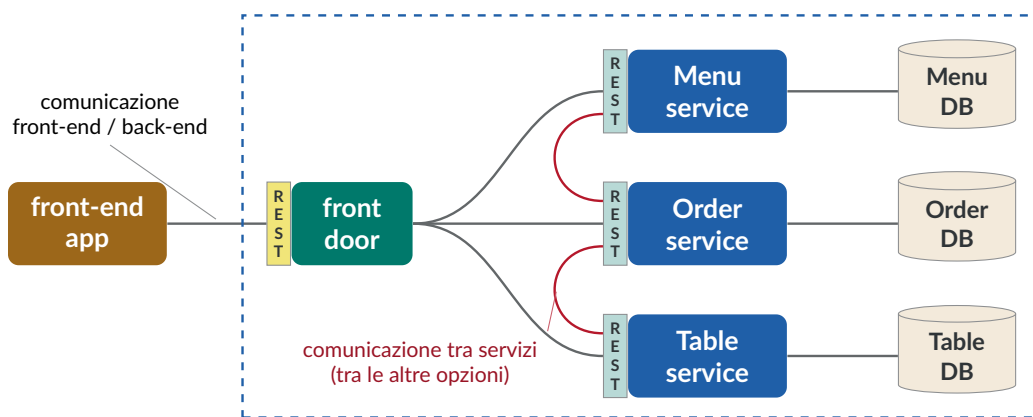


Figura 26: Dove si usa REST in un'architettura a microservizi: in giallo l'API esposta all'esterno dalla front door, in azzurro le API interne dei servizi

3.5.1 API esterne e API interne

Un servizio può esporre internamente, verso gli altri attori dell'architettura a microservizi, un'API che non deve essere necessariamente identica a quella esposta all'esterno.

Il rapporto tra l'API che si vede dal di fuori e quelle che i servizi espongono internamente dipende dal tipo di **front door**, perché la front door può fare un lavoro di traduzione. In particolare esistono due categorie di porte di ingresso.

Ingress È il caso più semplice: espone gli endpoint dei servizi tramite **syntactic routing**, cioè un routing basato sulla forma del path. Si possono mettere delle regole che riscrivono un po' la URL, però resta molto sintattico. L'API esterna è essenzialmente un sottoinsieme, o una mappatura diretta, delle API interne dei servizi. Gli ingress controller sono tipicamente componenti già pronti (*off-the-shelf*), che si configurano tramite regole.

API Gateway È una front door più sofisticata, che può contenere anche della **logica applicativa**: può avere del codice che dice di fare qualcosa (es. arriva una richiesta: prima contatta quel servizio lì, la risposta mandala al secondo, e al front-end manda la risposta del secondo). Può quindi comporre e orchestrare le API interne dei servizi in un'API esterna costruita apposta per dei consumatori specifici (es. web, mobile, servizi di partner).

In breve: un **ingress pubblica** ciò che c'è, mentre l'**API Gateway cura** che cosa viene esposto.

In sintesi

- Un'**operazione** è un contratto fatto di nome, input, output ed errori. Quando è remota bisogna identificarla, mandare gli input e riportare indietro i risultati attraverso la rete: servono la **serializzazione** dei dati e la gestione dei nuovi errori di rete.
- Un'API in **stile RPC** ragiona in termini di operazioni, input e output; il vero RPC nasconde la rete al programmatore, e per farlo ha bisogno di un framework.
- **REST** è uno stile architetturale basato sulle **risorse**: le URI esprimono nomi e non azioni, client e server si scambiano **rappresentazioni** delle risorse e le richieste sono **stateless**.
- I **verbi**: GET ottiene, POST crea, PUT sostituisce, DELETE cancella, PATCH aggiorna in parte. Solo GET non ha side effects; POST non è idempotente, PATCH dipende dal tipo di aggiornamento (Tabella 3).
- Gli **status code** hanno un significato preciso: 2XX successo, 3XX redirectione, 4XX errore del client, 5XX errore del server (Tabella 4).
- Con il **pragmatic REST** si usa solo JSON e, quando serve, si mettono dei verbi negli endpoint: alla fine del path, e rispettando la caratterizzazione dei verbi HTTP.
- Nelle MSOA REST si usa tra front-end e back-end e tra i servizi. Un **ingress pubblica** ciò che c'è; un **API Gateway cura** che cosa viene esposto.